TRƯỜNG ĐẠI HỌC CÔNG NGHỆ THÔNG TIN VÀ TRUYỀN THÔNG

Phát triển ứng dụng phân tán

Bài 2: RESTful API cho tương tác client–server · Lý thuyết → Catalog API → EduSubmit

Môn học: Phát triển ứng dụng phân tán

Mục tiêu:

HIỂU · PI 2.1

Truyền thông đồng bộ

Giải thích cơ chế request–response qua HTTP và cấu trúc một thông điệp.

HIỂU · PI 2.1

Nguyên tắc REST

Phân biệt tài nguyên, URI, phương thức HTTP, stateless và mã trạng thái.

HIỂU · PI 2.1

Thiết kế từ use case

Chọn URI, phương thức, mã trạng thái và DTO cho một yêu cầu nghiệp vụ.

VẬN DỤNG · PI 2.2

Viết Catalog API

Dùng MapGet/MapPost, nhận dữ liệu từ route, query, body và kiểm tra hợp lệ.

VẬN DỤNG · PI 2.2

Gọi API bằng HttpClient

Kiểm tra mã trạng thái trước khi đọc dữ liệu, đọc lỗi ApiError.

THỰC HÀNH · PI 2.3

EduSubmit

Hoàn thành 4 endpoint và client liệt kê → nộp → đọc lại.

Đặt vấn đề

1

Hai chương trình chạy ở hai nơi nói chuyện với nhau bằng gì?

2

Thiết kế một API bắt đầu từ đâu?

3

Làm sao máy khách biết yêu cầu thành công hay thất bại?

Hiểu = giải thích được · Biết = chạy lại được · Vận dụng = thiết kế được cho ứng dụng khác

PHẦN 1

Cơ chế truyền thông đồng bộ với RESTful API

Thông điệp chính: REST tổ chức request–response trên HTTP xoay quanh tài nguyên; hợp đồng API gồm URI, phương thức, mã trạng thái và DTO.

HTTP | Request–response | REST | Stateless | DTO

01

Thử thách 60 giây: cùng chức năng nộp bài, API nào dễ dùng nhất?

A

Yêu cầu

```
POST /doSubmit

{ "assignmentId": 1,
  "studentCode": "SV001",
  "repo": "https://…" }
```

Phản hồi

```
HTTP/1.1 200 OK
{ "result": "ok" }
```

B

Yêu cầu

```
POST /api/assignments/1/submissions

{ "studentCode": "SV001",
  "repositoryUrl": "https://…" }
```

Phản hồi

```
HTTP/1.1 201 Created
Location: /api/submissions/8313…
```

C

Yêu cầu

```
GET /api/submit?sv=SV001
        &repo=https://…

(không có body)
```

Phản hồi

```
HTTP/1.1 200 OK
{ "ok": false, "msg": "loi" }
```

NHIỆM VỤ *Chọn 1 phương án + 2 lý do. Trao đổi theo cặp trong 60 giây.*

Đáp án: B dễ dùng nhất vì hợp đồng rõ ở cả bốn yếu tố

A thiếu tín hiệu

URI chứa động từ **doSubmit**. Mọi kết quả đều **200**. Phải đọc body mới biết nộp cho bài tập nào.

B có hợp đồng rõ

URI là **danh từ**, POST nghĩa là **tạo**. **201 + Location** cho biết đã tạo và đọc lại ở đâu.

C sai ngữ nghĩa

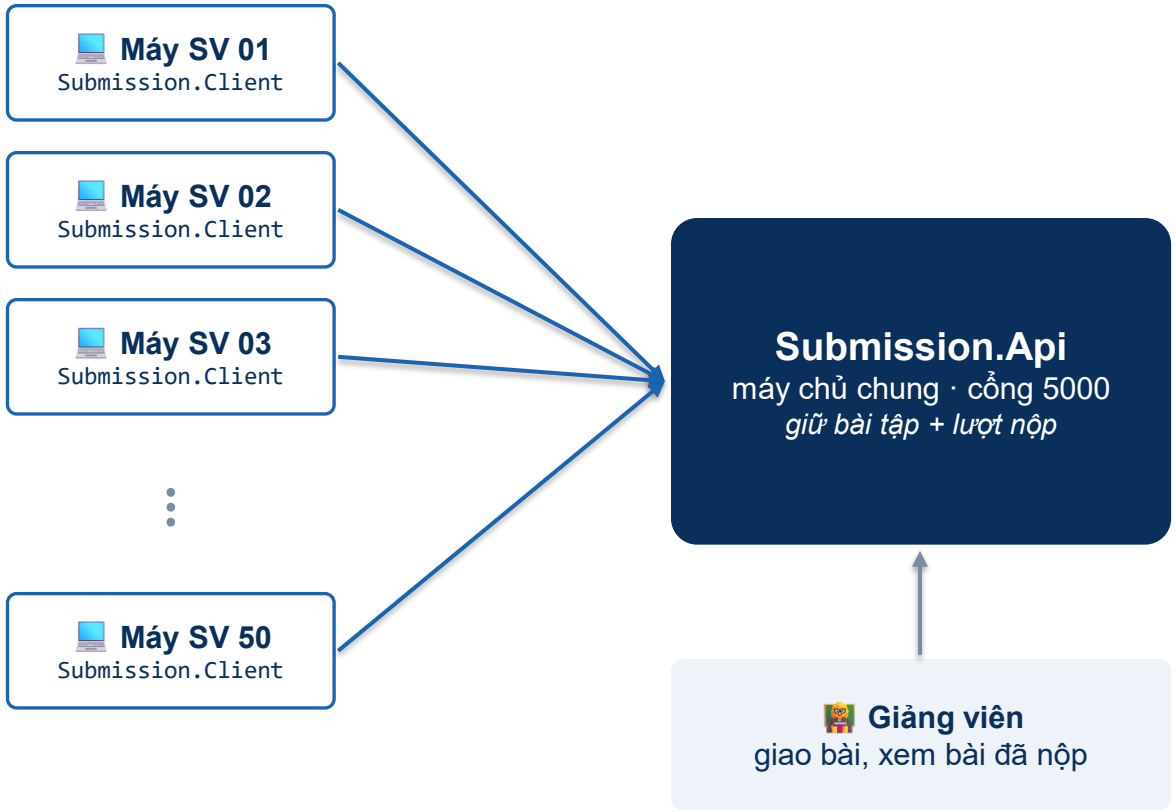
Dùng **GET** để thay đổi dữ liệu. Lỗi nhưng vẫn **200**. Dữ liệu lộ trên URL và nhật ký.

Bốn yếu tố của một hợp đồng API, cũng là bốn việc hôm nay học:

1 Tài nguyên (URI) → 2 Phương thức HTTP → 3 Mã trạng thái → 4 DTO (dữ liệu)

API tốt là API mà client đoán đúng cách dùng chỉ từ URI, phương thức và mã trạng thái.

Bài toán: 50 máy nộp bài vào một hệ thống chung



Nhưng khi hai chương trình ở hai nơi…

1 Không chung bộ nhớ

Client không gọi được hàm của server, không đọc được biến List của server.

→ **Phải gọi tên thứ cần dùng bằng một địa chỉ (URI).**

2 Chỉ gửi được thông điệp

Qua mạng chỉ có chuỗi byte, không có đối tượng C#.

→ **Hai bên phải thống nhất định dạng: HTTP + JSON.**

3 Có thể hỏng ở bất kỳ đâu

Sai mã bài tập, bài đã đóng, thiếu dữ liệu, mất mạng. Exception không bay qua mạng.

→ **Phản hồi phải tự nói: thành công hay thất bại, vì sao.**

Hệ phân tán = nhiều chương trình phối hợp với nhau CHỈ bằng thông điệp qua mạng.

Tách một chương trình làm hai: lời gọi hàm thành yêu cầu HTTP

TRƯỚC ĐÂY · ứng dụng một khối

1 chương trình = 1 tiến trình

Giao diện (console)

gọi hàm

`catalog.LayDanhSachSanPham()`

List<Product> trong bộ nhớ

Chung biến, chung bộ nhớ. Hàm luôn "có mặt".

tách làm hai

BÂY GIỜ · Catalog phân tán: 2 chương trình = 2 tiến trình

Catalog.Client

máy khách · console

Tiến trình 1 · Terminal 2

KHÔNG có dữ liệu

muốn biết gì phải hỏi

`GET /api/products`

`200 OK`
`+ JSON [ {...}, ... ]`

Catalog.Api

máy chủ · web

Tiến trình 2 · Terminal 1

GIỮ dữ liệu sản phẩm

lắng nghe ở cổng 5100

Mạng ở giữa: hai bên không chung biến, không chung bộ nhớ. Chỉ trao đổi được những gì gửi qua HTTP. Vì vậy cần **địa chỉ**, **hợp đồng** và cách xử lý **lỗi mạng**.

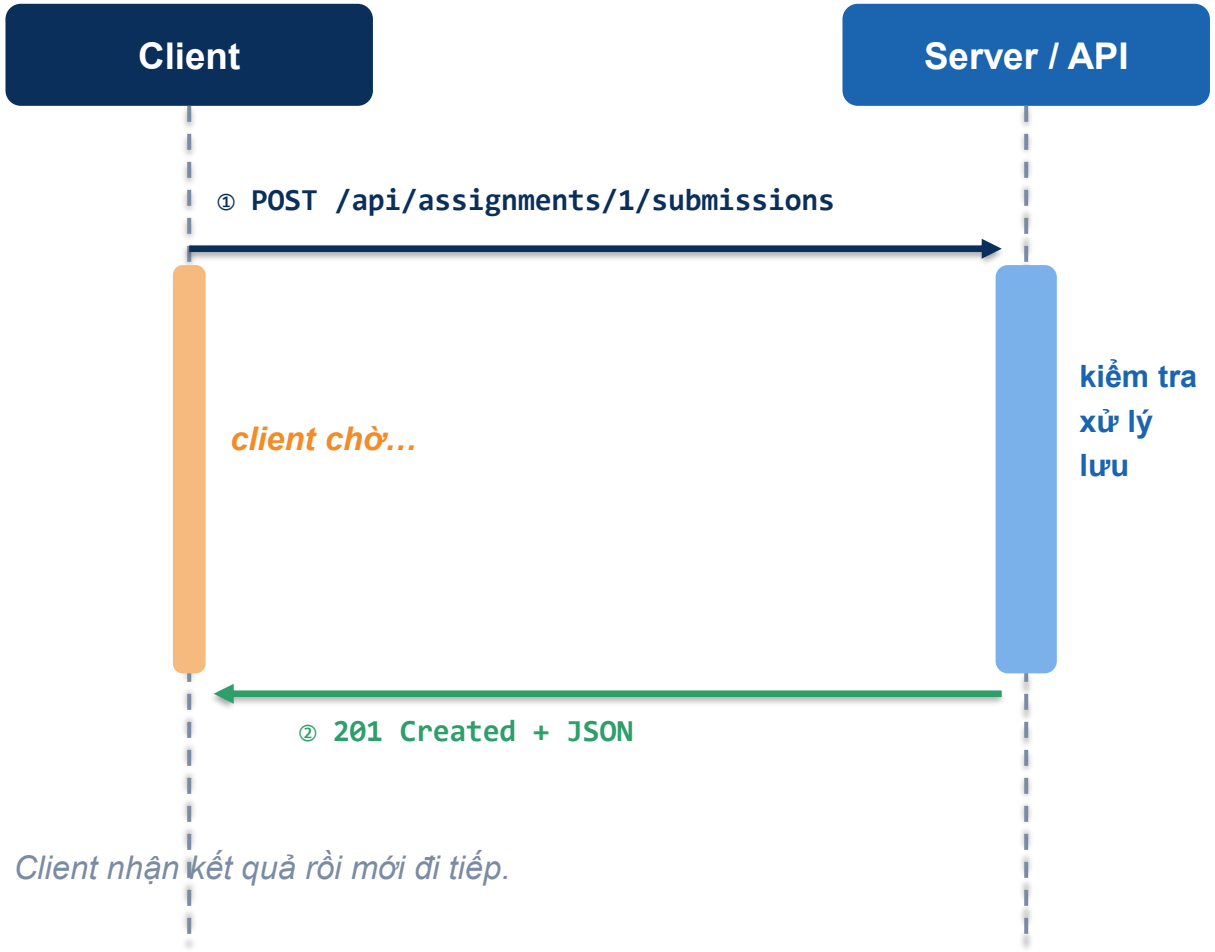
viết cả hai chương trình, rồi cho hai chương trình nói chuyện với nhau.

Gọi API giống gọi món ở quán ăn: thực đơn, phiếu gọi, món mang ra

Ở QUÁN ĂN		TRONG RESTful API
Khách gọi món	→	Client gửi **yêu cầu** (request)
Thực đơn: có món gì, gọi thế nào	→	**Hợp đồng API**: URI, phương thức, DTO
Phiếu gọi: bàn 5, 2 bát phở bò	→	`POST /api/orders` + body JSON
Bếp làm xong, mang món ra	→	**201 Created** + dữ liệu vừa tạo
"Món này hết rồi" / "phiếu ghi thiếu"	→	**404** Not Found / **400** Bad Request
Khách ngồi chờ đến khi có món	→	Truyền thông **đồng bộ**
Phiếu ghi đủ số bàn, ai phục vụ cũng được	→	**Stateless**: yêu cầu tự đủ thông tin

Thực đơn rõ thì khách gọi đúng món; hợp đồng API rõ thì client gọi đúng.

Truyền thông đồng bộ: client gửi yêu cầu rồi chờ phản hồi



RESTful API

Cách tổ chức giao tiếp đồng bộ client–server trên nền HTTP, **theo hướng tài nguyên**.

Ưu điểm

- Tài nguyên nghiệp vụ rõ ràng
- Phương thức đúng nghĩa
- Mã trạng thái và DTO nhất quán

Test

Mở **F12** → **Network** → **Fetch/XHR**: mỗi dòng là một cặp request–response.

Yêu cầu và phản hồi HTTP có cùng cấu trúc: dòng đầu, header, body

YÊU CẦU · client → server		PHẢN HỒI · server → client
`POST /api/assignments/1/submissions HTTP/1.1`	**Dòng đầu** phương thức + URI ↔ mã trạng thái	`HTTP/1.1 201 Created`

`Host: localhost:5000`
`Content-Type: application/json`
`Accept: application/json`
	Header
	siêu dữ liệu: kiểu nội dung, Location, xác thực…
		`Location: /api/submissions/8313f17c-…`
		`Content-Type: application/json`

(dòng trống)	**Dòng trống** ngăn cách header và body	*(dòng trống)*
`{ "studentCode": "SV001",` `  "repositoryUrl": "https://…/sv001" }`	**Body** dữ liệu JSON: DTO gửi lên ↔ DTO trả về	`{ "id": "8313f17c-…",` `  "assignmentId": 1, "status": "received" }`

thiết kế thông điệp ở tầng ứng dụng (HTTP); TCP/IP lo chia gói, gửi lại, định tuyến.

- Thiếu `Content-Type: application/json` khi gửi body → server trả **415** (sẽ thử ở phần 2).
- `Location` giúp client đọc lại tài nguyên vừa tạo mà không phải tự ghép đường dẫn.

Lời gọi qua mạng có thể không kết nối, lỗi, chậm hoặc bị lặp

Không kết nối được

server chưa chạy, sai cổng, mạng đứt

Server trả lỗi

4xx do yêu cầu sai, 5xx do server

Chờ quá lâu

phản hồi chậm hơn dự kiến, hết thời gian chờ

Yêu cầu bị lặp

gửi lại sau timeout: có tạo trùng không?

REST phù hợp khi…

- Thao tác có ranh giới rõ: **một yêu cầu, một phản hồi**
- Server xử lý xong trong thời gian client chấp nhận chờ
- Client cần kết quả ngay để đi tiếp

Cân nhắc cơ chế khác khi…

- Tác vụ kéo dài (chấm bài tự động) → **hàng đợi**, Bài 6
- Server cần đẩy cập nhật cho client → **SignalR**, Bài 3
- Nhiều dịch vụ cần biết một sự kiện → **pub/sub**, Bài 7

Soi thông điệp 3 phút: cặp yêu cầu–phản hồi này có 3 chỗ sai

Tình huống: client muốn **tạo lượt nộp** cho bài tập 1.

Yêu cầu client gửi

```
GET /api/assignments/1/submissions HTTP/1.1
Host: localhost:5000

{ "studentCode": "",
  "repositoryUrl": "https://…/sv001" }
```

Phản hồi server trả

```
HTTP/1.1 200 OK
Content-Type: application/json

{ "error": "Thieu ma sinh vien" }
```

Soi dòng đầu

Phương thức có đúng với việc “tạo”?

Soi header

Gửi body JSON thì cần header gì?

Soi mã trạng thái

Dữ liệu thiếu mà trả 200 có hợp lý?

NHIỆM VỤ ***Cá nhân 2 phút: khoanh 3 chỗ sai và viết cách sửa. 1 phút đối chiếu theo cặp.***

Đáp án: sai phương thức, thiếu Content-Type, lỗi mà vẫn trả 200

1 GET → POST

Tạo dữ liệu thì dùng **POST**. GET chỉ để đọc, không được thay đổi dữ liệu.

2 Thiếu Content-Type

Gửi body JSON phải có `Content-Type: application/json`, nếu không server trả **415**.

3 200 → 400

Thiếu mã sinh viên là lỗi do yêu cầu: trả **400 + ApiError**, không trả 200.

Yêu cầu đã sửa

```
POST /api/assignments/1/submissions HTTP/1.1
Host: localhost:5000
Content-Type: application/json

{ "studentCode": "",
  "repositoryUrl": "https://…/sv001" }
```

Phản hồi đúng

```
HTTP/1.1 400 Bad Request
Content-Type: application/json

{ "code": "INVALID_SUBMISSION",
  "message": "Thieu ma sinh vien…" }
```

Đọc thông điệp HTTP theo thứ tự: dòng đầu → header → body → mã trạng thái.

REST là phong cách kiến trúc với các ràng buộc, không phải giao thức

1 Client–server

Tách giao diện khỏi lưu trữ, xử lý; hai bên tiến hóa độc lập.

2 Stateless

Mỗi yêu cầu chứa đủ thông tin để server hiểu và xử lý.

3 Giao diện thống nhất

Tài nguyên có URI; thao tác qua phương thức HTTP và biểu diễn JSON.

4 Có thể cache

Phản hồi cho biết có được lưu tạm hay không, giảm tải cho server.

5 Phân tầng

Client không cần biết phía sau là proxy, cân bằng tải hay server thật.

Cách giải: REST là bộ quy ước trả lời đúng 3 khó khăn đó

Khó khăn	Câu hỏi thiết kế	Quy ước REST	Hiện ra trong code EduSubmit
① Không chung bộ nhớ	① Hệ thống có những "thứ" gì?	Tài nguyên = danh từ, mỗi thứ một URI	`/api/assignments/{assignmentId}`
	② Người dùng muốn làm gì với nó?	Phương thức HTTP = động từ (GET đọc, POST tạo)	`app.MapGet(…)` `app.MapPost(…)`
② Chỉ gửi thông điệp	③ Dữ liệu gửi/nhận có hình dạng gì?	JSON theo khuôn DTO làm hợp đồng	`record CreateSubmissionRequest(…)`
③ Có thể hỏng	④ Hỏng thì báo thế nào?	Mã trạng thái + một dạng lỗi chung	`Results.NotFound(new ApiError(…))`
(gom cả ba)	⑤ Client dùng theo thứ tự nào?	Client xét mã trạng thái trước khi đọc dữ liệu	`Submission.Client/Program.cs`

5 câu hỏi thiết kế không phải để học thuộc: mỗi câu sinh ra từ một khó khăn của hệ phân tán.

URI: đường dẫn chỉ tài nguyên, chuỗi truy vấn chọn tập con

https://	api.example.edu	/api/assignments/42/submissions	?status=late&page=1
lược đồ	máy chủ	đường dẫn → tài nguyên	chuỗi truy vấn → lọc, phân trang

Đọc URI EduSubmit theo lớp ý nghĩa

URI	Ý nghĩa	Loại
`/api/assignments`	tập tài nguyên bài tập	tập hợp
`/api/assignments/42`	bài tập có mã 42	một phần tử
`/api/assignments/42/submissions`	các lượt nộp thuộc bài tập 42	tài nguyên con
`/api/submissions/8313f17c-…`	một lượt nộp cụ thể	một phần tử
`/api/submissions?studentCode=SV001`	lọc lượt nộp của SV001	tập con (query)

Quy ước: danh từ số nhiều, chữ thường, phân cấp cha/con bằng "/"; tiền tố /api nhóm các endpoint công khai.

URI hay URL? Phần đường dẫn là URI, cả địa chỉ là URL



Mọi URL đều là URI.

Không phải URI nào cũng là URL.

	Dùng URI khi nói về…	Dùng URL khi nói về…
Ý nghĩa	tên của tài nguyên trong API	địa chỉ đầy đủ để kết nối tới máy chủ
Trong code	`MapGet("/api/assignments/{assignmentId:int}")` `Results.Created($"/api/submissions/{id}")`	`--urls http://localhost:5000` `BaseAddress = new Uri(apiBaseUrl)`
Ví dụ khác	header Location, bảng hợp đồng API, cây tài nguyên	@api trong requests.http, thanh địa chỉ trình duyệt

Thiết kế API: nói URI. Cấu hình để kết nối: nói URL.

URI là danh từ; động từ để phương thức HTTP đảm nhận

✕ Kiểu "gọi hàm" (tránh) | ✓ Kiểu tài nguyên (nên dùng)

Trước	Sau	Ý nghĩa
`/getAssignments`	`GET /api/assignments`	Đọc danh sách bài tập
`/getAssignmentById?id=42`	`GET /api/assignments/42`	Đọc một bài tập
`/doSubmit`	`POST /api/assignments/42/submissions`	Tạo lượt nộp cho bài tập 42
`/createProduct`	`POST /api/products`	Tạo sản phẩm
`/api/assignment-service/do-create`	`POST /api/assignments`	Tạo bài tập (theo nghiệp vụ, không theo tên lớp)

Mẹo: thấy động từ trong URI (get, create, do…) → hỏi "tài nguyên ở đây là gì?"

Phương thức HTTP: hai tính chất an toàn và lũy đẳng

Phương thức	Ý nghĩa thường gặp	Ví dụ trong EduSubmit	An toàn	Lũy đẳng
`GET`	Lấy biểu diễn của tài nguyên	Danh sách bài tập, chi tiết một lượt nộp	✓	✓
`POST`	Tạo tài nguyên con / yêu cầu xử lý mới	Tạo lượt nộp mới cho một bài tập	✗	✗
`PUT`	Thay thế toàn bộ trạng thái tài nguyên	Cập nhật toàn bộ thông tin một bài tập	✗	✓
`PATCH`	Cập nhật một phần tài nguyên	Cập nhật hạn nộp	✗	✗
`DELETE`	Xóa hoặc hủy liên kết tài nguyên	Xóa bản nháp bài nộp	✗	✓

An toàn (safe)

Không yêu cầu server thay đổi trạng thái nghiệp vụ.

Lũy đẳng (idempotent)

Gửi cùng yêu cầu nhiều lần, hiệu ứng như gửi một lần.

Chọn: phương thức nào, gửi lại có sao không?

#	Tình huống	Phương thức?	Gửi lại 2 lần có an toàn? (lũy đẳng)
1	SV xem danh sách bài tập		
2	SV nộp bài cho bài tập 1		
3	GV thay toàn bộ thông tin bài tập 2		
4	GV chỉ sửa hạn nộp của bài tập 2		
5	SV xóa bản nháp số 7		

NHIỆM VỤ ***Cặp đôi: điền 2 cột trống. Giơ tay khi xong, cặp nhanh nhất đọc đáp án.***

Đáp án: chỉ POST và PATCH không đảm bảo an toàn khi gửi lại

#	Tình huống	Phương thức	Gửi lại 2 lần
1	SV xem danh sách bài tập	**GET**	✓ chỉ đọc
2	SV nộp bài cho bài tập 1	**POST**	✗ có thể tạo 2 lượt nộp
3	GV thay toàn bộ thông tin bài tập 2	**PUT**	✓ thay cùng nội dung
4	GV chỉ sửa hạn nộp của bài tập 2	**PATCH**	✗ không đảm bảo
5	SV xóa bản nháp số 7	**DELETE**	✓ xóa rồi vẫn là đã xóa

Hai câu hỏi mỗi khi chọn phương thức: thao tác là gì? gửi lại có gây tác dụng phụ không?

Stateless: mỗi yêu cầu tự mang đủ thông tin để server xử lý

✕ Dựa vào phiên trong bộ nhớ server

1. Yêu cầu 1 tới **server A**: “chọn bài tập 1”
   A ghi nhớ lựa chọn trong bộ nhớ
2. Yêu cầu 2 bị cân bằng tải chuyển tới **server B**: “nộp bài”
3. B không biết bài tập nào → **lỗi**

✓ Mỗi yêu cầu tự đủ thông tin

1. `POST /api/assignments/1/submissions`
   + token xác thực trong header
2. Server **A hay B** đều xử lý được
3. Thêm server để chịu tải mà không cần chia sẻ phiên

NGỘ NHẬN THƯỜNG GẶP

Stateless **không** có nghĩa “server không lưu dữ liệu”. Bài tập, lượt nộp vẫn lưu trong CSDL; server chỉ **không nhớ cuộc hội thoại** giữa hai yêu cầu.

Mã trạng thái là một phần của hợp đồng API, không phải chi tiết phụ

2xx · Thành công

200	**OK** thành công, có dữ liệu
201	**Created** đã tạo tài nguyên mới
202*	**Accepted** đã nhận, xử lý sau
204	**No Content** thành công, không body

4xx · Lỗi do yêu cầu

400	**Bad Request** thiếu hoặc sai dữ liệu
401	**Unauthorized** chưa xác thực
403	**Forbidden** không có quyền
404	**Not Found** không có tài nguyên
409*	**Conflict** xung đột trạng thái
415*	**Unsupported Media** sai Content-Type

5xx · Lỗi phía server

500	**Internal Error** lỗi không lường trước
503*	**Unavailable** quá tải, bảo trì

*Không dấu *: giáo trình mục 2.1.3. * Bổ sung ngoài giáo trình: IETF RFC 9110 – HTTP Semantics (2022), mục 15.*

DTO là hợp đồng dữ liệu ở biên hệ thống, tách khỏi mô hình bên trong

Client

client → server · DTO yêu cầu

```
record CreateSubmissionRequest(
    string StudentCode,
    string RepositoryUrl);
```

server → client · DTO phản hồi

```
record SubmissionDto(
    Guid Id, int AssignmentId,
    string StudentCode,
    string RepositoryUrl,
    string Status,
    DateTimeOffset ReceivedAt);
```

biên API

Bên trong server · Entity (minh họa)

```
class Submission {
  Guid Id
  int AssignmentId
  string StudentCode
  string RepositoryUrl
  string Status
  DateTimeOffset ReceivedAt
  string GraderNote   // nội bộ
  decimal RawScore    // nội bộ
}
```

Không lộ nội bộ

Client chỉ thấy điều cần thấy.

Client gửi đúng phần được phép

Không gửi Id, Status.

Bên trong đổi, hợp đồng giữ nguyên

Client không vỡ khi server sửa.

Entity minh họa để so sánh; ở Bài 2 dữ liệu còn lưu trong List<…> nên DTO cũng là dạng lưu trữ.

Khung 5 câu hỏi dùng cho mọi API: trả lời xong là có bản thiết kế

	CÂU HỎI	KẾT QUẢ	TRONG CODE C#
①	Hệ thống có những **danh từ** nào? Cái nào thuộc cái nào?	Tài nguyên, quan hệ cha–con	`/api/<cha>/{id}/<con>`
②	Người dùng muốn **làm gì** với từng danh từ?	Xem, tạo, sửa, xóa	`MapGet` `MapPost` `MapDelete`
③	Máy khách **gửi gì, nhận gì**? Trường nào máy chủ tự sinh?	DTO vào / DTO ra	`record ...Request` / `...Dto`
④	Có thể **sai ở đâu**? Sai thì nói gì?	Mã trạng thái + mã lỗi	`Results.NotFound(new ApiError(..))`
⑤	Ai gọi, **theo thứ tự nào**, lỗi thì làm gì?	Kịch bản máy khách	`HttpClient` + kiểm tra `StatusCode`

Thiết kế = trả lời 5 câu. Code = dịch câu trả lời sang C#.

Từ điển nhanh: 8 thuật ngữ của Bài 2 nói bằng lời đơn giản

Tài nguyên

“Thứ” mà API quản lý: bài tập, lượt nộp, sản phẩm.

URI

Địa chỉ của tài nguyên, ví dụ /api/assignments/1.

Phương thức HTTP

Động từ: GET đọc, POST tạo, PUT thay, PATCH sửa, DELETE xóa.

Mã trạng thái

Con số báo kết quả: 2xx ổn, 4xx client sai, 5xx server lỗi.

DTO

“Mẫu đơn” dữ liệu gửi đi và nhận về qua API.

Stateless

Server không nhớ yêu cầu trước; mỗi yêu cầu tự đủ thông tin.

Lũy đẳng

Gửi lại nhiều lần, kết quả như gửi một lần.

Hợp đồng API

Thỏa thuận chung: URI + phương thức + DTO + mã trạng thái.

PHẦN 2

Catalog API

Thông điệp chính: mỗi bước kết thúc bằng một kết quả chạy được; viết một endpoint, kiểm thử ngay endpoint đó.

dotnet CLI | MapGet / MapPost | DTO | HttpClient | requests.http

02

Mỗi bước gồm đúng 4 việc: gõ, hiểu, chạy, tự kiểm tra



GÕ

Gõ lệnh hoặc chép đoạn code vào đúng vị trí slide chỉ.



HIỂU

Đọc thẻ giải thích từng dòng. Không hiểu dòng nào thì hỏi ngay.



CHẠY

Chạy lệnh test, so kết quả với khung “kết quả thật”.



KIỂM TRA

Thanh xanh lá cuối bước: đạt hết mới sang bước sau.

Sau buổi học

Hiểu

Giải thích vì sao Catalog là hệ phân tán thu nhỏ: 2 chương trình, 2 tiến trình, nói bằng HTTP + JSON.

Biết

Tự tạo solution bằng dotnet, viết 4 endpoint, viết máy khách HttpClient, test bằng curl.exe và requests.http.

Vận dụng

Tự thêm endpoint mới (lọc, xóa, kiểm tra) hoặc đổi sang miền khác theo đúng khung 10 bước.

Bản đồ tư duy: Catalog phân tán gồm 6 nhánh của buổi học

Catalog phân tán

Tiến trình — Bước 1

Catalog.Api (máy chủ)
Catalog.Client (máy khách)

Giao tiếp — Bước 2–3

HTTP: phương thức + URI
dữ liệu dạng JSON

Địa chỉ — Bước 2, 7

URL http://localhost:5100
--urls ở máy chủ, BaseAddress ở máy khách

Hợp đồng — Bước 3–6

endpoint · DTO
mã trạng thái · ApiError

Máy khách — Bước 7–8

HttpClient gửi yêu cầu
đọc mã trước, body sau

Lỗi & trạng thái — Bước 9

máy chủ tắt → lỗi kết nối
dữ liệu ở máy chủ, mất khi tắt

Cuối buổi, tự vẽ lại bản đồ này và ghi một ví dụ code vào mỗi nhánh.

Đích đến: hợp đồng API của Catalog gồm 4 endpoint

Phương thức	URI	Thành công	Lỗi có thể gặp
GET	/	200 + {service, status}	–
GET	`/api/products`	200 + danh sách	–
GET	`/api/products/{id}`	200 + một sản phẩm	404 PRODUCT_NOT_FOUND 404 rỗng nếu id không phải số
POST	`/api/products`	201 + Location + sản phẩm mới	400 INVALID_PRODUCT 415 nếu thiếu Content-Type

1

URI là danh từ

Không có /getProducts hay /createProduct. Hành động nằm ở GET/POST.

2

Mã trạng thái là hợp đồng

Máy khách đọc mã trước, rồi mới đọc body theo đúng kiểu.

3

Mọi lỗi cùng một dạng

ApiError { code, message }: máy khách chỉ cần một cách xử lý lỗi.

Chưa viết dòng code nào, hai bên đã biết sẽ nói với nhau những gì.

Lộ trình 10 bước: mỗi bước kết thúc bằng một kết quả chạy được

Bước 1

Tạo solution, 2 project

`Build succeeded`

Bước 2

Chạy khung API

`200`

Bước 3

DTO + GET danh sách

`200 + 3 sản phẩm`

Bước 4

GET chi tiết

`200 · 404 · 405`

Bước 5

POST tạo mới

`201 · 400 · 415`

Bước 6

requests.http

`test cả bộ`

Bước 7

Client nấc 1

`in danh sách`

Bước 8

Client nấc 2–4

`đọc 201, 400`

Bước 9

Thí nghiệm phân tán

`lỗi kết nối`

Bước 10

Git + tự kiểm tra

`phiếu 10 ô`

Xanh dương: khung · Cam: API · Xanh lá: máy khách · Navy: thí nghiệm và nộp bài

Chuẩn bị: kiểm tra SDK và cài 2 extension trước khi bắt đầu

chuẩn bị

Terminal trong VS Code (Ctrl+`)

```
PS> dotnet --version
10.0.114
```

`dotnet`

Công cụ dòng lệnh của .NET: tạo, build, chạy project.

`--version`

In phiên bản SDK. Cần dạng 10.0.xxx. Báo “not recognized” = chưa cài.

Extension VS Code

C# Dev Kit: hiểu code C#, gợi ý, báo lỗi.
REST Client: gửi yêu cầu HTTP từ tệp .http (Bước 6).

Quy ước trong slide

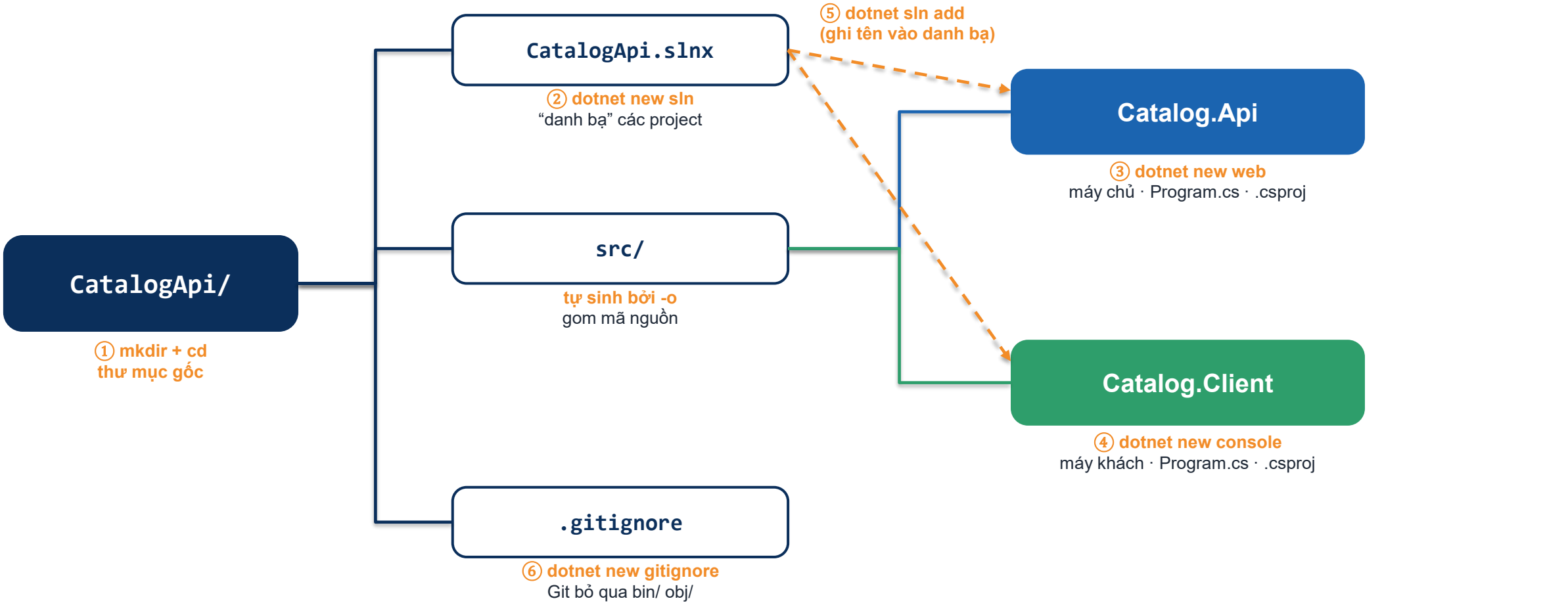
Lệnh viết cho PowerShell, mỗi lệnh 1 dòng.
PS> là dấu nhắc, KHÔNG gõ. Chữ cam đậm = dòng mới/quan trọng.
Dữ liệu trong code viết không dấu.

Thư mục làm bài

Mở VS Code ở thư mục làm bài, ví dụ D:\PTUDPT\Bai2. Mọi lệnh Bước 1 gõ ở đó.

✓ KIỂM TRA **dotnet --version ra 10.0.xxx và đã cài C# Dev Kit, REST Client**

Bước 1: nhìn đích đến trước, mỗi lệnh mọc thêm một nhánh



⑦ dotnet build ở thư mục gốc: solution là danh bạ, build cả hai nhánh một lần.

Bước 1.1–1.2: tạo thư mục, đứng vào đó, tạo tệp solution

Bước 1

⌨ Gõ (ở thư mục làm bài)

```
mkdir CatalogApi
cd CatalogApi
```

`mkdir CatalogApi`

make directory: tạo thư mục mới tên CatalogApi.

`cd CatalogApi`

change directory: đi vào thư mục đó. Từ giờ MỌI lệnh gõ ở đây. Đứng sai thư mục là lỗi số 1.

⌨ Gõ tiếp

```
dotnet new sln -n CatalogApi

The template "Solution File" was
created successfully.
```

`dotnet new`

Tạo thứ gì đó mới từ một mẫu (template).

`sln`

Mẫu “Solution File”: tệp danh bạ liệt kê các project của bài.

`-n CatalogApi`

name: tên tệp là CatalogApi.slnx (.NET 10 dùng .slnx).

✓ KIỂM TRA **Gõ dir: thấy tệp CatalogApi.slnx trong thư mục CatalogApi**

Bước 1.3–1.4: tạo hai project, một máy chủ và một máy khách

Bước 1

Máy chủ

```
dotnet new web -n Catalog.Api -o src/Catalog.Api

The template "ASP.NET Core Empty"
was created successfully.
Restore succeeded.
```

Máy khách (gõ trên MỘT dòng)

```
dotnet new console -n Catalog.Client
   -o src/Catalog.Client

The template "Console App" was created...
```

`web`

Mẫu ASP.NET Core Empty: máy chủ web rỗng, chỉ 4 câu lệnh. Mọi endpoint đều tự viết.

`-n Catalog.Api`

Tên project, cũng là tên tệp Catalog.Api.csproj.

`-o src/Catalog.Api`

output: đặt project trong src/ để tách mã nguồn khỏi các tệp khác.

`console`

Mẫu ứng dụng dòng lệnh: máy khách chỉ gửi yêu cầu và in kết quả, không cần giao diện.

`Ý quan trọng`

Api và Client là HAI chương trình riêng → khi chạy là hai tiến trình riêng. Đó là hệ phân tán thu nhỏ.

✓ KIỂM TRA **Có thư mục src/Catalog.Api và src/Catalog.Client, mỗi thư mục có Program.cs**

Bước 1.5–1.6: ghi project vào danh bạ, tạo .gitignore, build thử

Bước 1

Ghi danh

```
dotnet sln add src/Catalog.Api/Catalog.Api.csproj
dotnet sln add src/Catalog.Client/Catalog.Client.csproj

Project `src/Catalog.Api/...` added to the solution.
```

`dotnet sln add`

Ghi đường dẫn project vào CatalogApi.slnx. dotnet new KHÔNG tự ghi.

`Vì sao cần`

Có danh bạ thì một lệnh dotnet build build được cả hai.

.gitignore và build

```
dotnet new gitignore
dotnet build

Build succeeded.
    0 Warning(s)
    0 Error(s)
```

`dotnet new gitignore`

Tạo .gitignore để Git bỏ qua bin/ và obj/ (hàng nghìn tệp sinh ra khi build, không được nộp).

`dotnet build`

Build khi chưa viết code: nếu lỗi thì chắc chắn do cài đặt, không phải do code.

Nếu build lỗi NETSDK1045: máy chưa có .NET 10. Nếu “Couldn't find a project”: terminal đang đứng sai thư mục, cd về CatalogApi.

✓ KIỂM TRA **dotnet build báo Build succeeded, 0 Error(s)**

Trên đĩa là một cây thư mục, khi chạy là hai tiến trình riêng

Bước 1

TRÊN ĐĨA · lúc viết code

```
CatalogApi/
├── CatalogApi.slnx
└── src/
    ├── Catalog.Api/
    └── Catalog.Client/
```

dotnet run

KHI CHẠY · hai terminal, hai tiến trình

Tiến trình 1 · Terminal 2

Catalog.Client

```
dotnet run
 --project
 src/Catalog.Client
```

yêu cầu HTTP

phản hồi HTTP

Tiến trình 2 · Terminal 1

Catalog.Api :5100

```
dotnet run
 --project src/Catalog.Api
 --urls
 http://localhost:5100
```

Chạy Api trước

Client gửi mà chưa có ai lắng nghe → Connection refused.

Chỉ nói qua HTTP

Client không gọi được hàm hay đọc biến products của Api.

Tách được ra hai máy

Đổi localhost thành địa chỉ máy khác là chạy được, code không đổi.

Một solution, hai project, hai tiến trình: hệ phân tán thu nhỏ.

Bước 2: Program.cs mẫu chỉ có 4 câu lệnh, mỗi câu một vai trò

💡 **src/Catalog.Api/Program.cs (mẫu sinh sẵn)**

```
var builder = WebApplication.CreateBuilder(args);
var app = builder.Build();

app.MapGet("/", () => "Hello World!");

app.Run();
```

Liên hệ ứng dụng một khối:

MapGet giống một câu switch lớn: “nếu là GET và đường dẫn là / thì gọi hàm này”.

Tập hợp các Map... gọi là bảng định tuyến.

`CreateBuilder(args)`

Chuẩn bị máy chủ web Kestrel, cấu hình, log. args: nhận tham số dòng lệnh như --urls.

`builder.Build()`

Đóng gói cấu hình thành ứng dụng app, sẵn sàng khai báo endpoint.

`MapGet("/", () => ...)`

Ánh xạ yêu cầu GET / tới một hàm xử lý. () => ... là lambda: hàm không tên.

`app.Run()`

Bắt đầu lắng nghe, đứng chờ tới khi Ctrl+C. Mọi Map... phải viết TRƯỚC dòng này.

Một endpoint = phương thức + đường dẫn + hàm xử lý.

Bước 2.2: sửa endpoint “/” để trả JSON có mã 200

Bước 2

⌨ Xóa dòng Hello World, thay bằng khối cam

```
var builder = WebApplication.CreateBuilder(args);
var app = builder.Build();

// GET / : kiểm tra máy chủ đang chạy
app.MapGet("/", () => Results.Ok(new {
    service = "Catalog.Api",
    status = "running"
}));

app.Run();
```

Kết quả sẽ thấy ở bước sau:

```
{"service":"Catalog.Api","status":"running"}
```

`// GET / : ...`

Chú thích, máy bỏ qua. Ghi endpoint ở đầu khối cho dễ tìm.

`Results.Ok( ... )`

Trả mã 200 OK kèm dữ liệu. Ghi rõ “Ok” để người đọc biết ý định.

`new { service = ..., status = ... }`

Đối tượng vô danh: không cần khai báo lớp. ASP.NET tự đổi thành JSON.

`Tên trong JSON`

Tự đổi sang camelCase: service, status.

NHIỆM VỤ **Lưu tệp (Ctrl+S). Chưa chạy, sang slide sau.**

Bước 2.3–2.4: chạy máy chủ ở cổng 5100, rồi gọi thử bằng curl.exe

Bước 2

Terminal 1 · máy chủ (gõ trên MỘT dòng)

```
dotnet run --project src/Catalog.Api
   --urls http://localhost:5100

info: Now listening on:
      http://localhost:5100
info: Application started.
      Press Ctrl+C to shut down.
```

Terminal 2 · bấm + để mở terminal mới

```
curl.exe -i http://localhost:5100/

HTTP/1.1 200 OK
Content-Type: application/json;
              charset=utf-8
Server: Kestrel

{"service":"Catalog.Api","status":"running"}
```

`--project src/Catalog.Api`

Thư mục gốc có 2 project, phải chỉ rõ chạy cái nào.

`--urls http://localhost:5100`

Ép cổng 5100. Không ghi thì mẫu chọn cổng ngẫu nhiên (vd 5007), máy khách không biết gọi đâu.

`curl.exe -i`

Gõ đủ .exe (trong PowerShell, curl là lệnh khác). -i: in cả mã trạng thái và header.

Sửa code máy chủ?

Terminal 1: Ctrl+C, bấm ↑ để gọi lại lệnh, Enter. Không khởi động lại = code cũ.

✓ KIỂM TRA **Terminal 1 có “Now listening on: http://localhost:5100”; curl.exe ra 200 OK + JSON**

Trước khi thêm code: Program.cs của máy chủ có 5 vùng, theo đúng thứ tự

Bước 3

1 Khởi tạo — có sẵn

```
var builder = ...; var app = builder.Build();
```

2 Dữ liệu mẫu — Bước 3

```
var products = new List<ProductDto> { ... };
```

3 Các endpoint — Bước 2–5

```
app.MapGet(...); app.MapPost(...);
```

4 Chạy — có sẵn

```
app.Run();
```

5 Khai báo kiểu — Bước 3, 5

```
record ProductDto(...); record ApiError(...);
```

Vì sao record ở cuối tệp?

Program.cs dùng câu lệnh cấp cao nhất: không có class Program, không có Main.

Quy tắc C#: câu lệnh viết trước, khai báo kiểu (record, class) viết sau.

Viết sai vị trí thì sao?

record đặt giữa tệp → lỗi CS8803: Top-level statements must precede namespace and type declarations.

MapGet đặt sau app.Run() → endpoint không có, gọi vào 404.

Mỗi khi thêm code, tự hỏi: đoạn này thuộc vùng nào?

Bước 3.1: khai báo DTO, hình dạng dữ liệu đi qua mạng

Cuối tệp src/Catalog.Api/Program.cs

```
// DTO: hình dạng dữ liệu đi qua mạng  (vùng 5, sau app.Run();)
record ProductDto(int Id, string Name, string Description, decimal Price);
record ApiError(string Code, string Message);
```

record

Kiểu dữ liệu gọn: một dòng là có lớp với thuộc tính, hàm khởi tạo, so sánh theo giá trị. Hợp để làm DTO.

(int Id, string Name, ...)

Mỗi tham số trong ngoặc thành một thuộc tính chỉ đọc: Id, Name, Description, Price.

decimal Price

Tiền dùng decimal, không dùng double, để không sai số làm tròn (0,1 + 0,2 ≠ 0,3 với double).

ApiError(Code, Message)

Dạng lỗi chung cho MỌI lỗi. Code cho máy đọc (PRODUCT_NOT_FOUND), Message cho người đọc.

DTO = Data Transfer Object: kiểu chỉ để chở dữ liệu qua mạng.

Bước 3.2–3.3: thêm dữ liệu mẫu và endpoint danh sách

Bước 3

⌨ Vùng 2: ngay sau var app = builder.Build();

```
// Dữ liệu mẫu trong bộ nhớ
var products = new List<ProductDto>
{
    new(1, "Giao trinh Lap trinh phan tan", "Giao trinh chinh thuc", 150000),
    new(2, "Tai lieu RESTful API", "Huong dan thiet ke API", 90000),
    new(3, "Bai tap thuc hanh .NET", "Bo bai tap kem loi giai", 120000)
};
```

⌨ Vùng 3: sau khối MapGet("/"), trước app.Run();

```
// GET /api/products : danh sách sản phẩm
app.MapGet("/api/products", () => Results.Ok(products));
```

`new List<ProductDto> { ... }`

Danh sách sản phẩm, đóng vai “CSDL” tạm thời.

`new(1, "...", "...", 150000)`

C# tự suy ra kiểu ProductDto. Thứ tự: Id, Name, Description, Price.

Lưu ý

Dữ liệu nằm trong bộ nhớ tiến trình máy chủ: tắt là mất (Bước 9).

`"/api/products"`

URI là danh từ số nhiều: “tập hợp sản phẩm”. Tiền tố /api tách phần dữ liệu.

`MapGet · Results.Ok(products)`

Động từ GET: chỉ đọc. Trả 200 + cả danh sách thành mảng JSON.

NHIỆM VỤ **Lưu, Ctrl+C ở Terminal 1, chạy lại máy chủ. Sang slide sau để test.**

Bước 3.4: GET /api/products trả 200 và mảng JSON 3 sản phẩm

Bước 3

▶ Terminal 2 · kết quả thật (thực tế nằm trên một dòng)

```
curl.exe -i http://localhost:5100/api/products

HTTP/1.1 200 OK
Content-Type: application/json; charset=utf-8

[{"id":1,"name":"Giao trinh Lap trinh phan tan","description":"Giao trinh chinh thuc","price":150000},
 {"id":2,"name":"Tai lieu RESTful API","description":"Huong dan thiet ke API","price":90000},
 {"id":3,"name":"Bai tap thuc hanh .NET","description":"Bo bai tap kem loi giai","price":120000}]
```

C# → JSON

Id, Name trong C# thành id, name trong JSON (camelCase).

List → mảng

List<ProductDto> thành mảng JSON [{...}, {...}].

decimal → số

150000 là số, không có dấu nháy.

✓ KIỂM TRA **curl.exe ra 200 OK và đủ 3 sản phẩm**

Bước 4: chi tiết một sản phẩm, tìm thấy thì 200, không thấy thì 404

Bước 4

⌨ Vùng 3: sau khối GET /api/products

```
// GET /api/products/{id:int} : chi tiết một sản phẩm
app.MapGet("/api/products/{id:int}", (int id) => {
    var product = products.FirstOrDefault(p => p.Id == id);
    return product is null
        ? Results.NotFound(new ApiError("PRODUCT_NOT_FOUND",
                                        "Khong tim thay san pham."))
        : Results.Ok(product);
});
```

Liên hệ ứng dụng một khối

Một khối: LaySanPham(2) trả null hoặc ném exception.

Phân tán: GET /api/products/2 trả mã 404 + ApiError. Mã trạng thái thay cho exception, vì exception không đi qua mạng được.

`{id:int}`

{id}: tham số tuyến, phần thay đổi của URI. :int = chỉ khớp khi là số nguyên.

`(int id) =>`

ASP.NET lấy chữ "2" trên URI, đổi thành số 2, đưa vào id (liên kết từ route).

`FirstOrDefault(p => p.Id == id)`

Tìm phần tử đầu tiên có Id bằng id. Không có → null.

`cond ? A : B`

Toán tử ba ngôi: null → A (404 + ApiError), ngược lại → B (200 + sản phẩm).

`Results.NotFound(...)`

404 kèm body lỗi đúng dạng chung: máy khách biết VÌ SAO không có.

NHIỆM VỤ **Lưu, khởi động lại máy chủ, sang slide sau để test 4 ca.**

Bước 4: bốn ca test cho thấy ai trả lời, handler hay framework

Bước 4

Lệnh (Terminal 2)	Kết quả thật	Ai trả lời, vì sao
`curl.exe -i http://localhost:5100/api/products/1`	`200 OK + {"id":1,"name":"Giao trinh..."}`	Handler: tìm thấy
`curl.exe -i http://localhost:5100/api/products/999`	`404 + {"code":"PRODUCT_NOT_FOUND",...}`	Handler: chạy, không tìm thấy, tự trả 404 có body
`curl.exe -i http://localhost:5100/api/products/abc`	`404, Content-Length: 0 (rỗng)`	Framework: abc không phải số, không khớp tuyến, handler KHÔNG chạy
`curl.exe -i -X POST http://localhost:5100/api/products`	`405 Method Not Allowed` `Allow: GET`	Framework: URI có, nhưng chưa có động từ POST (Bước 5)

Suy nghĩ: cả /999 và /abc đều 404. Khi viết máy khách, phân biệt hai trường hợp bằng gì? (Gợi ý: nhìn body.)

✓ KIỂM TRA **Đủ 4 ca đúng như bảng; giải thích được vai trò của :int**

Bước 5.1: DTO đầu vào không có Id, vì Id do máy chủ sinh

Bước 5

Vùng 5: thêm dòng cam

```
record ProductDto(int Id, string Name, string Description, decimal Price);  // đầu ra
record CreateProductRequest(string Name, string? Description, decimal Price);  // đầu vào
record ApiError(string Code, string Message);
```

ProductDto · máy chủ GỬI ĐI

Có Id: máy khách cần biết sản phẩm số mấy để đọc lại.

Description bắt buộc (máy chủ luôn điền, ít nhất là "").

CreateProductRequest · máy khách GỬI LÊN

KHÔNG có Id: máy khách không được tự chọn Id, tránh hai người gửi cùng số.

string? Description: dấu ? = được phép null, mô tả là tùy chọn.

Quy tắc: dữ liệu đi VÀO và dữ liệu đi RA là hai hợp đồng khác nhau, nên dùng hai DTO khác nhau. Máy chủ quyết định những gì máy khách không được quyết định (Id, thời điểm tạo, trạng thái…).

NHIỆM VỤ **Thêm dòng cam vào cuối Program.cs máy chủ, lưu tệp.**

Bước 5.2: POST nhận body JSON, kiểm tra hợp lệ ngay ở cửa vào

Bước 5

⌨ Vùng 3: sau khối GET {id:int} (phần 1/2)

```
// POST /api/products : tạo mới từ body JSON
app.MapPost("/api/products", (CreateProductRequest request) => {
    // Kiểm tra tại biên hệ thống
    if (string.IsNullOrWhiteSpace(request.Name) || request.Price <= 0) {
        return Results.BadRequest(new ApiError(
            "INVALID_PRODUCT",
            "Ten san pham khong duoc de trong va gia phai lon hon 0."));
    }
    // ... phần tạo sản phẩm: slide sau ...
});
```

Biên hệ thống

Là chỗ dữ liệu từ bên ngoài đi vào. Máy chủ không bao giờ tin dữ liệu máy khách gửi: máy khách có thể viết sai, hoặc cố tình gửi bậy.

Ở ứng dụng một khối, dữ liệu được kiểm tra ở form; ở đây phải kiểm tra lại ở máy chủ.

`MapPost("/api/products")`

Cùng URI với danh sách, động từ POST = “thêm một phần tử vào tập hợp”.

`(CreateProductRequest request)`

Kiểu phức tạp → ASP.NET đọc từ body JSON và tạo request (liên kết từ body).

`IsNullOrWhiteSpace(Name)`

Đúng khi tên null, rỗng, hoặc chỉ toàn dấu cách.

`|| Price <= 0`

Hoặc giá không dương. Một điều sai là từ chối.

`Results.BadRequest(...)`

400: lỗi do dữ liệu máy khách gửi, body nói rõ lỗi gì.

NHIỆM VỤ **Gõ phần 1. Chưa chạy được vì handler chưa trả gì khi dữ liệu đúng.**

Bước 5.2: máy chủ sinh Id, lưu, trả 201 kèm Location

Bước 5

Phần 2/2: thay dòng "// ... phần tạo sản phẩm ..."

```
    // (tiếp sau khối if, trước dấu }); )
    var product = new ProductDto(
        Id: products.Max(p => p.Id) + 1,
        Name: request.Name.Trim(),
        Description: request.Description?.Trim() ?? "",
        Price: request.Price);

    products.Add(product);
    return Results.Created($"/api/products/{product.Id}", product);
```

Phản hồi 201 trông thế này

```
HTTP/1.1 201 Created
Location: /api/products/4
{"id":4,"name":"Lap trinh gRPC",...}
```

Location chỉ đường cho máy khách: muốn đọc lại thì GET tới đây.

`Max(p => p.Id) + 1`

Id lớn nhất cộng 1. "Id:" là đối số có tên, đọc rõ giá trị nào vào đâu.

`Name.Trim()`

Bỏ dấu cách thừa hai đầu tên.

`Description?.Trim() ?? ""`

?. : null thì không gọi Trim (tránh lỗi). ?? : null thì dùng "".

`products.Add(product)`

Thêm vào danh sách trong bộ nhớ.

`Results.Created(uri, product)`

201 + header Location = URI của sản phẩm mới + body. $"..." thay {product.Id} bằng số.

NHIỆM VỤ **Lưu, khởi động lại máy chủ. Toàn bộ Program.cs: Phụ lục A tài liệu Word.**

Bước 5.3: gửi POST bằng curl.exe, body lấy từ tệp JSON cho khỏi lỗi nháy

Bước 5

Tệp sp-hop-le.json (thư mục gốc CatalogApi)

```
{ "name": "Lap trinh gRPC",
  "description": "Tai lieu bai 5", "price": 80000 }
```

Tệp sp-sai.json

```
{ "name": "San pham loi",
  "description": "", "price": -1 }
```

▶ Terminal 2 · ba lệnh, mỗi lệnh một dòng

```
curl.exe -i -X POST http://localhost:5100/api/products -H "Content-Type: application/json" -d "@sp-hop-le.json"
curl.exe -i -X POST http://localhost:5100/api/products -H "Content-Type: application/json" -d "@sp-sai.json"
curl.exe -i -X POST http://localhost:5100/api/products -d "@sp-hop-le.json"
```

`-X POST`

Dùng phương thức POST thay vì GET mặc định.

`-H "Content-Type: application/json"`

Header báo máy chủ: body gửi kèm là JSON.

`-d "@tệp.json"`

Body lấy từ tệp. Dấu @ = “đọc tệp”. Gõ JSON trực tiếp trong PowerShell rất dễ hỏng dấu nháy.

NHIỆM VỤ **Chạy cả ba lệnh, ghi lại mã trạng thái của từng lệnh trước khi sang slide sau.**

Bước 5.4: ba ca POST cho ba mã trạng thái, và ai là người từ chối

Bước 5

Ca	Kết quả thật	Ai trả lời, vì sao

① Hợp lệ (sp-hop-le.json)	`201 Created`
	`Location: /api/products/4`
	`{"id":4,"name":"Lap trinh gRPC",...}`
		Handler: Id = 3 + 1 = 4

② Giá âm (sp-sai.json)	`400 Bad Request` `{"code":"INVALID_PRODUCT",...}`	Handler: khối if từ chối
③ Thiếu Content-Type	`415 Unsupported Media Type` `(đoạn chữ dài, không phải ApiError)`	Framework: không biết đọc body là gì, chặn trước handler

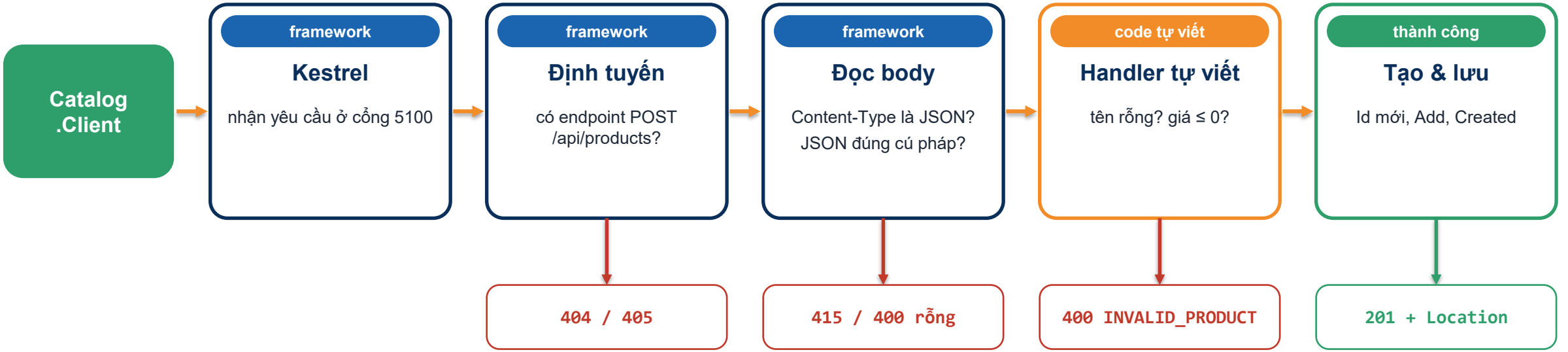
Gọi lại GET /api/products: giờ có 4 sản phẩm. **Đoạn chữ dài ở ca ③:** chế độ Development in chi tiết lỗi cho lập trình viên; không cần đọc hết.

✓ KIỂM TRA **Đủ 201 (có Location), 400, 415; giải thích được 400 khác 415 ở ai từ chối**

Một yêu cầu POST qua 4 trạm kiểm tra rồi mới được tạo, trạm nào cũng có thể từ chối

Bước 5

Đi từ trái sang phải. Bị từ chối ở trạm nào thì phản hồi quay về máy khách ngay, các trạm sau không chạy.



Đọc mã là biết lỗi ở đâu: 404/405: gõ sai URI hoặc phương thức. 415: quên header. 400 rỗng: JSON hỏng. 400 có ApiError: dữ liệu sai luật nghiệp vụ. Chỉ trạm thứ 4 là code tự viết.

Mã trạng thái là bản đồ chỉ chỗ hỏng.

Bước 6: tệp requests.http gom mọi ca test, bấm Send Request là chạy

Bước 6

⌨ Tệp requests.http ở thư mục gốc CatalogApi (trích)

```
@api = http://localhost:5100

### Danh sách sản phẩm → 200
GET {{api}}/api/products

### Sản phẩm không tồn tại → 404 + ApiError
GET {{api}}/api/products/999

### Tạo sản phẩm hợp lệ → 201 Created + Location
POST {{api}}/api/products
Content-Type: application/json

{ "name": "Lap trinh gRPC", "description": "Tai lieu bai 5",
  "price": 80000 }

### ... đủ 8 ca: xem Bước 6 tài liệu Word
```

`@api = ...`

Biến. {{api}} được thay bằng địa chỉ: đổi cổng chỉ sửa một chỗ.

`###`

Ngăn cách các yêu cầu. Chữ sau ### là tiêu đề, ghi luôn mã mong đợi.

`Dòng GET/POST`

Dòng yêu cầu: phương thức + URL.

`Dòng ngay dưới`

Header, ví dụ Content-Type.

`DÒNG TRỐNG rồi JSON`

Body. Bắt buộc có một dòng trống giữa header và body.

✓ KIỂM TRA **8 yêu cầu trong requests.http đều cho đúng mã ghi ở tiêu đề**

Bước 7: máy khách cần biết địa chỉ máy chủ và một HttpClient

Bước 7

src/Catalog.Client/Program.cs: XÓA HẾT rồi gõ

```
using System.Net;
using System.Net.Http.Json;

var apiBaseUrl = args.Length > 0 ? args[0] : "http://localhost:5100";

using var http = new HttpClient {
    BaseAddress = new Uri(apiBaseUrl)
};
```

Khác biệt đầu tiên so với gọi hàm

Gọi hàm: không cần biết hàm "ở đâu".

Gọi qua mạng: máy khách PHẢI biết địa chỉ máy chủ (URL). Sai địa chỉ = không gọi được (thử ở Bước 9).

`using System.Net;`

Dùng HttpStatusCode: mã trạng thái có tên (Created, BadRequest…).

`using System.Net.Http.Json;`

Hàm tiện ích gửi/nhận JSON: GetFromJsonAsync, PostAsJsonAsync…

`args.Length > 0 ? args[0] : "..."`

Có tham số khi chạy thì dùng, không thì dùng địa chỉ mặc định.

`using var http = new HttpClient`

Đối tượng gửi yêu cầu HTTP. using var: tự giải phóng kết nối khi xong.

`BaseAddress`

URL gốc. Các lời gọi sau chỉ cần URI ngắn /api/products.

NHIỆM VỤ **Gõ xong 8 dòng này, sang slide sau gõ tiếp.**

Bước 7: gọi GET, JSON tự đổi thành List<ProductDto>

Bước 7

⌨ Gõ tiếp ngay dưới

```
// Bước 1: lấy danh sách sản phẩm
var products = await http.GetFromJsonAsync<List<ProductDto>>("/api/products");
Console.WriteLine("Danh sach san pham:");
foreach (var p in products ?? []) {
    Console.WriteLine($"  [{p.Id}] {p.Name} - {p.Price:N0} VND");
}

// cuối tệp
record ProductDto(int Id, string Name, string Description, decimal Price);
```

▶ Terminal 2 (máy chủ vẫn chạy ở Terminal 1)

```
dotnet run --project src/Catalog.Client

Danh sach san pham:
  [1] Giao trinh Lap trinh phan tan - 150,000 VND
  [2] Tai lieu RESTful API - 90,000 VND
  [3] Bai tap thuc hanh .NET - 120,000 VND
```

`await GetFromJsonAsync<List<ProductDto>>`

Gửi GET, chờ phản hồi, đổi JSON thành List<ProductDto>. await: chờ mạng mà không “đơ”.

`products ?? []`

null thì dùng danh sách rỗng, vòng lặp không lỗi.

`{p.Price:N0}`

Số có dấu phân cách nghìn: 150,000 (hoặc 150.000 tùy máy).

`record ProductDto(...)`

BẢN SAO DTO của máy chủ. Hai chương trình không chung code, chỉ chung hợp đồng: tên phải khớp JSON.

✓ KIỂM TRA **Máy khách in danh sách lấy từ máy chủ; giải thích vì sao phải khai báo lại ProductDto**

Bước 8.2: gửi POST và kiểm tra mã trạng thái TRƯỚC khi đọc body

Bước 8

⌨ Sau vòng foreach, trước các record · thêm 2 record ở cuối tệp (xem dưới)

```
var createRequest = new CreateProductRequest(
    "Nhap mon REST API", "Sach tham khao", 120000);

var createResponse = await http.PostAsJsonAsync("/api/products", createRequest);

if (createResponse.StatusCode == HttpStatusCode.Created) {
    var created = await createResponse.Content.ReadFromJsonAsync<ProductDto>();
    Console.WriteLine($"\nDa tao san pham moi: [{created?.Id}] {created?.Name}");
    Console.WriteLine($"Doc lai tai: {createResponse.Headers.Location}");
} else {
    var error = await createResponse.Content.ReadFromJsonAsync<ApiError>();
    Console.WriteLine($"\nLoi: {error?.Code} - {error?.Message}");
    return;
}
```

⌨ Cuối tệp, cạnh ProductDto

```
record CreateProductRequest(string Name, string? Description, decimal Price);
record ApiError(string Code, string Message);
```

`PostAsJsonAsync(uri, obj)`

Đổi obj thành JSON, gửi POST, tự gắn Content-Type. Trả về CẢ phản hồi, chưa đọc body.

`StatusCode == Created`

Đọc mã trước: 201 thì body là ProductDto, mã khác thì body là ApiError. Hai hình dạng khác nhau.

`Content.ReadFromJsonAsync<T>()`

Đọc body, đổi JSON thành kiểu T.

`Headers.Location`

Đọc header Location máy chủ gửi kèm 201.

`created?.Id · \n · return;`

?. tránh lỗi null · \n xuống dòng · return dừng chương trình khi lỗi.

Đọc mã trạng thái trước, chọn kiểu đọc body sau.

Bước 8.3–8.4: đọc lại chi tiết, rồi cố ý gửi sai để xem lỗi 400

Bước 8

⌨ Nấc 3: sau khối if/else

```
// Bước 3: đọc lại chi tiết một sản phẩm
var detail = await http.GetFromJsonAsync<ProductDto>("/api/products/1");
Console.WriteLine($"\nChi tiet san pham 1: {detail?.Name}");
```

`GetFromJsonAsync<ProductDto>`

GET một sản phẩm. Hàm này NÉM exception nếu mã không phải 2xx: chỉ dùng khi chắc chắn tồn tại.

⌨ Nấc 4: ngay dưới nấc 3

```
// Bước 4: cố ý gửi dữ liệu sai để quan sát lỗi 400
var invalidResponse = await http.PostAsJsonAsync("/api/products",
    new CreateProductRequest("", "", -1));

Console.WriteLine($"\nThu gui du lieu sai: {(int)invalidResponse.StatusCode}
{invalidResponse.StatusCode}");
var invalidError = await invalidResponse.Content.ReadFromJsonAsync<ApiError>();
Console.WriteLine($"{invalidError?.Code}: {invalidError?.Message}");
```

`new CreateProductRequest("", "", -1)`

Tên rỗng, giá âm: chắc chắn bị từ chối.

`(int)invalidResponse.StatusCode`

Đổi tên mã sang số: BadRequest → 400. In cả hai.

`ReadFromJsonAsync<ApiError>()`

Mọi lỗi cùng dạng ApiError nên một kiểu là đủ.

Vì sao cố ý gửi sai? Máy khách tốt phải xử lý được cả khi máy chủ nói “không”. Nấc 4 chứng minh máy khách đọc được lỗi 400 thành câu dễ hiểu, thay vì chết hoặc in rác.

NHIỆM VỤ **Lưu. Toàn bộ Program.cs máy khách: Phụ lục B tài liệu Word.**

Bước 8.5: chạy hệ thống hai tiến trình và đọc kết quả 4 nấc

Bước 8

Terminal 1 · máy chủ

```
Ctrl+C rồi chạy lại để dữ liệu về 3 sản phẩm:
dotnet run --project src/Catalog.Api
   --urls http://localhost:5100
```

Terminal 2 · máy khách (một dòng)

```
dotnet run --project src/Catalog.Client
   -- http://localhost:5100
```

-- (hai gạch)

Mọi thứ SAU -- được chuyển cho chương trình đang chạy, vào mảng args. http://localhost:5100 thành args[0].

▶ Kết quả thật

```
Danh sach san pham:
  [1] Giao trinh Lap trinh phan tan - 150,000 VND
  [2] Tai lieu RESTful API - 90,000 VND
  [3] Bai tap thuc hanh .NET - 120,000 VND
                                  // nấc 1
Da tao san pham moi: [4] Nhap mon REST API
Doc lai tai: /api/products/4
                                  // nấc 2
Chi tiet san pham 1: Giao trinh Lap trinh phan tan
                                  // nấc 3
Thu gui du lieu sai: 400 BadRequest
INVALID_PRODUCT: Ten san pham khong duoc
  de trong va gia phai lon hon 0.
                                  // nấc 4
```

✓ KIỂM TRA **Máy khách in đủ 4 nấc như khung bên phải**

Bước 9: bốn thí nghiệm, mỗi thí nghiệm một bài học về phân tán

Bước 9

1 Chạy máy khách lần 2

(máy chủ vẫn chạy)

Kết quả thật

```
Danh sách có 4 sản phẩm, tạo mới
ra [5], Doc lai tai:
/api/products/5
```

Bài học

Trạng thái nằm ở máy chủ, không ở máy khách.

2 Tắt máy chủ

(Ctrl+C ở Terminal 1), chạy máy khách

Kết quả thật

```
Unhandled exception.
HttpRequestException: Connection
refused (localhost:5100)
```

Bài học

Lỗi chỉ có ở phân tán: bên kia có thể vắng mặt.

3 Chạy lại máy chủ

rồi GET /api/products

Kết quả thật

```
Lại chỉ còn 3 sản phẩm
```

Bài học

Dữ liệu trong bộ nhớ mất khi tiến trình dừng.

4 Sai cổng

```
dotnet run --project
src/Catalog.Client --
http://localhost:5999
```

Kết quả thật

```
Connection refused
(localhost:5999)
```

Bài học

Hai bên phải thống nhất địa chỉ (URL).

NHIỆM VỤ **Ghi kết quả quan sát vào cột cuối bảng Bước 9 trong tài liệu Word**

Thử thách: gọi qua mạng khác gọi hàm ở những điểm nào?

thử thách

A

Gọi hàm LaySanPham(2) không bao giờ bị “Connection refused”. Vì sao gọi GET /api/products/2 thì có thể?

B

Máy khách chạy 2 lần thấy dữ liệu khác nhau, dù code máy khách không đổi. Dữ liệu đó “sống” ở đâu?

C

Nếu máy khách khai báo record ProductDto(int Id, string Ten, ...) thì chương trình có lỗi biên dịch không? Chạy ra gì?

NHIỆM VỤ **Thảo luận theo cặp 3 phút, viết câu trả lời ra giấy. Đáp án ở slide sau.**

Đáp án: ba điều chỉ hệ phân tán mới có

thử thách

A **Bên kia có thể vắng mặt**

Hàm nằm cùng tiến trình nên luôn có. Máy chủ là tiến trình khác, có thể chưa chạy, đã tắt, sai cổng, mạng đứt.

B **Trạng thái nằm ở máy chủ**

Danh sách products sống trong bộ nhớ của Catalog.Api. Máy khách chỉ hỏi; ai hỏi cũng thấy cùng dữ liệu.

C **Hợp đồng không được trình biên dịch kiểm tra**

Không lỗi biên dịch. Ten luôn null vì JSON có “name” chứ không có “ten”. Hai bên chỉ “hứa” với nhau qua hợp đồng.

Vắng mặt · trạng thái ở xa · hợp đồng chỉ là lời hứa: ba chủ đề của cả môn học.

Bước 10: commit sau mỗi bước và điền phiếu tự kiểm tra

Bước 10

⌨ Gợi ý commit (mỗi bước một commit)

```
git add .
git commit -m "Buoc 1: tao solution CatalogApi"
git commit -am "Buoc 2: GET / tra JSON"
git commit -am "Buoc 3: DTO va GET /api/products"
git commit -am "Buoc 4: GET /api/products/{id}"
git commit -am "Buoc 5: POST /api/products"
git add .
git commit -m "Buoc 6: requests.http"
git commit -am "Buoc 7-8: Catalog.Client"
```

`-am` = commit mọi tệp đã theo dõi có thay đổi. Tệp MỚI (requests.http) phải git add trước. `git status` không được thấy bin/ hay obj/.

#	Yêu cầu	Mong đợi	Tự đánh giá
1	GET /	200	
2	GET /api/products	200, 3 phần tử	
3	GET /api/products/1	200	
4	GET /api/products/999	404 + ApiError	
5	GET /api/products/abc	404 rỗng	
6	POST hợp lệ	201 + Location	
7	POST giá âm	400 + ApiError	
8	POST thiếu Content-Type	415	
9	Máy khách chạy	đủ 4 nấc	
10	Tắt máy chủ, chạy máy khách	lỗi kết nối	

✓ KIỂM TRA **Đủ 10 ô ✓ và có ít nhất 7 commit. Nộp theo hướng dẫn nộp bài của lớp.**

PHẦN 3 · ÁP DỤNG

Áp dụng sang EduSubmit

Thông điệp chính: giữ nguyên khuôn của Catalog, đổi tên tài nguyên, thêm tài nguyên con và quy tắc nghiệp vụ.

5 câu hỏi | tài nguyên con | 3 cổng kiểm tra | ApiError | Location

03

Từ Catalog sang EduSubmit: giữ khuôn, đổi tên, thêm tài nguyên con

Catalog API (Phần 2)	EduSubmit lát cắt 2 (Phần 3)
`ProductDto(Id, Name, Description, Price)`	`AssignmentDto(Id, Title, Description, IsOpen)`
`CreateProductRequest(Name, Description, Price)`	`CreateSubmissionRequest(StudentCode, RepositoryUrl)`
`GET /api/products` · `GET /api/products/{id:int}`	`GET /api/assignments` · `GET /api/assignments/{assignmentId:int}`
`POST /api/products`	`POST /api/assignments/{assignmentId}/submissions` (tài nguyên con)
(không có)	`GET /api/submissions/{submissionId:guid}` (đọc lại theo Location)
`PRODUCT_NOT_FOUND / INVALID_PRODUCT`	`ASSIGNMENT_NOT_FOUND / ASSIGNMENT_CLOSED / INVALID_SUBMISSION`
`Cổng 5100 · CatalogApi.slnx`	`Cổng 5000 · RestfulClientServer.slnx`

Khuôn giữ nguyên; phần mới là tài nguyên con, thứ tự kiểm tra nghiệp vụ và đọc lại theo Location.

Thử thách 5 phút: trả lời 5 câu hỏi cho hệ thống nộp bài

① **Hệ thống nộp bài có những danh từ nào?**

Câu trả lời của lớp: ...

② **Sinh viên muốn làm gì?**

Câu trả lời của lớp: ...

③ **Khi nộp bài, sinh viên gửi những gì? Có được tự ghi trạng thái "đã chấm" không?**

Câu trả lời của lớp: ...

④ **Nộp bài có thể hỏng vì những lý do nào?**

Câu trả lời của lớp: ...

⑤ **Một sinh viên dùng ứng dụng theo thứ tự nào?**

Câu trả lời của lớp: ...

NHIỆM VỤ ***Thảo luận theo cặp 3 phút, sau đó từng cặp trả lời một câu.***

Đáp án: 5 câu trả lời biến thành bảng hợp đồng API của EduSubmit

Phương thức	URI	Thành công	Lỗi có thể gặp
GET	`/api/assignments`	200 danh sách	–
GET	`/api/assignments/{assignmentId}`	200 một bài tập	404 `ASSIGNMENT_NOT_FOUND`
POST	`/api/assignments/{assignmentId}/submissions`	201 + `Location`	404 `ASSIGNMENT_NOT_FOUND` · 400 `ASSIGNMENT_CLOSED` · 400 `INVALID_SUBMISSION`
GET	`/api/submissions/{submissionId}`	200 lượt nộp	404 `SUBMISSION_NOT_FOUND`
POST trùng *	như trên	–	409 `DUPLICATE_SUBMISSION`
DELETE *	`/api/submissions/{submissionId}`	204 không body	404 `SUBMISSION_NOT_FOUND`

** Mở rộng ở bước 9, ngoài giáo trình. Mọi lỗi trả cùng dạng `{ "code": "...", "message": "..." }`*

Chưa viết dòng code nào, API đã được thiết kế xong.

Lượt nộp là tài nguyên con: assignmentId chuyển từ body lên tuyến

Bài 1 · chung chung

```
POST /api/submissions
{ "assignmentId": 1, "studentCode": "SV001", … }
```

Phải đọc body mới biết nộp cho bài tập nào.

Bài 2 · tài nguyên con

```
POST /api/assignments/1/submissions
{ "studentCode": "SV001", "repositoryUrl": "…" }
```

Nhìn URI là biết: tạo lượt nộp thuộc bài tập 1.

Cây tài nguyên



Vì sao đọc lại qua /api/submissions/{id}?

- Id lượt nộp là **Guid duy nhất** toàn hệ thống
- Client chỉ cần URL trong header **Location**
- Không cần biết lượt nộp thuộc bài tập nào

Hai tiến trình nói chuyện qua một hợp đồng

Submission.Client

console · tiến trình 1 · máy sinh viên

⑤ Kịch bản 4 bước
xem → nộp → đọc lại → thử lỗi

HttpClient
đóng gói yêu cầu, gửi, nhận phản hồi

record bản sao
AssignmentDto, SubmissionDto, ApiError…

Yêu cầu HTTP: method + URI + JSON →

← **Phản hồi HTTP: mã trạng thái + JSON**

③ HỢP ĐỒNG API (hai bên cùng tuân theo)

```
GET  /api/assignments                → 200
GET  /api/assignments/{id}           → 200 | 404
POST /api/assignments/{id}/submissions
                                     → 201 | 400 | 404
GET  /api/submissions/{id}           → 200 | 404
```

`requests.http` = máy khách thứ hai,
dùng chung hợp đồng, không cần sửa server

Submission.Api

Minimal API · tiến trình 2 · cổng 5000

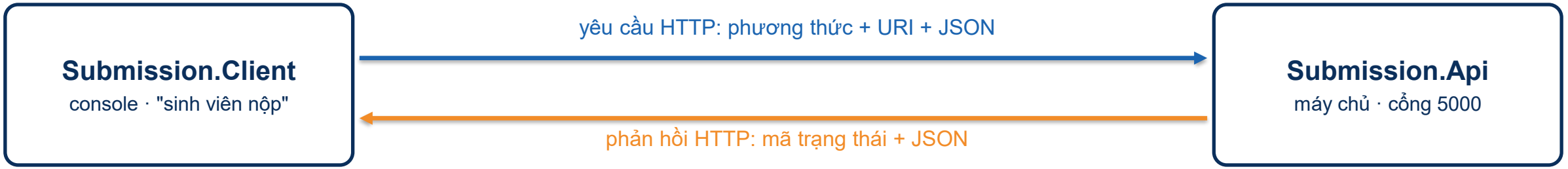
①② Endpoint
MapGet / MapPost: URI + method

④ Luật nghiệp vụ
tồn tại? còn mở? đủ dữ liệu?

Dữ liệu trong RAM
List assignments · List submissions

Hai bên chỉ biết nhau qua hợp đồng: sửa bên trong một bên, bên kia không cần biết.

Bản đồ 9 bước: mỗi bước trả lời một câu hỏi trong khung



Bước 1–2 — nền
Dựng dự án, chạy máy chủ

Bước 3 — ① ③
GET danh sách + DTO

Bước 4 — ② ④
GET một bài tập

Bước 5 — ② ③ ④
POST nộp bài

Bước 6 — ② ④
GET lượt nộp

Bước 7 — ⑤
Máy khách

Bước 8 — ④
requests.http

Bước 9 — ② ④
409, 204

Client và Api là hai chương trình, hai tiến trình: một hệ phân tán thu nhỏ.

Bước 1: ba lệnh tạo solution, project máy chủ và project máy khách

nền móng

```
mkdir RestfulClientServer   ;   cd RestfulClientServer
```

```
dotnet new sln -n RestfulClientServer
```

Để làm gì: Tạo tệp `RestfulClientServer.slnx`.
Vì sao: Solution gom nhiều project: `dotnet build` một lần build cả hai. .NET 10 mặc định tạo `.slnx`.

```
dotnet new web -n Submission.Api
  -o src/Submission.Api
```

Để làm gì: Tạo máy chủ từ mẫu **ASP.NET Core Empty**.
Vì sao: Mẫu `web` chỉ 5 dòng, không controller, Swagger. `-n` đặt tên, `-o` đặt vào `src/`.

```
dotnet new console -n Submission.Client
  -o src/Submission.Client
```

Để làm gì: Tạo máy khách là ứng dụng console.
Vì sao: Máy khách chỉ gửi yêu cầu và in kết quả. **Hai chương trình, hai tiến trình riêng.**

*Lệnh `-o` xuống dòng ở đây chỉ để vừa slide; trên PowerShell gõ trên **một dòng**.*

Bước 1: gắn project vào solution, tạo .gitignore, build thử khung rỗng

nền móng

```
dotnet sln add src/Submission.Api/Submission.Api.csproj
dotnet sln add src/Submission.Client/Submission.Client.csproj
```

Để làm gì: Đăng ký hai project vào solution.
Vì sao: Không gắn thì `dotnet build` ở thư mục gốc không biết build gì.

```
dotnet new gitignore
```

Để làm gì: Tạo `.gitignore` chuẩn .NET.
Vì sao: Không đẩy `bin/`, `obj/` lên repo nộp bài.

```
dotnet build
```

Để làm gì: Biên dịch cả solution.
Vì sao: Kiểm tra khung trước khi viết code. Mong đợi: `0 Error(s)`.

```
RestfulClientServer/
├── RestfulClientServer.slnx
├── .gitignore
└── src/
    ├── Submission.Api/     (máy chủ)
    └── Submission.Client/  (máy khách)
```

Mẹo

`code .` mở thư mục trong VS Code; C# Dev Kit tự nhận solution.

Bước 2: chạy máy chủ ở cổng 5000 cố định, rồi gọi thử bằng curl.exe

nền móng

Terminal 1 · máy chủ

```
dotnet run --project src/Submission.Api
  --urls http://localhost:5000
```

```
info: Now listening on: http://localhost:5000
info: Application started.
      Press Ctrl+C to shut down.
```

Terminal 2 · gọi thử

```
curl.exe -i http://localhost:5000/
```

```
HTTP/1.1 200 OK
Content-Type: application/json; charset=utf-8

{"service":"Submission.Api","status":"running"}
```

`--project`

Thư mục gốc có 2 project: chỉ rõ chạy cái nào

`--urls ...:5000`

Mẫu chọn cổng ngẫu nhiên; cố định để cả lớp dùng chung

`curl.exe`

PowerShell 5.1: `curl` là bí danh `Invoke-WebRequest`

`-i`

In cả **dòng trạng thái** và header, không chỉ body

Test ngay: thấy 200 OK là khung máy chủ đã sống.

Bước 3: khai báo DTO, dữ liệu mẫu và endpoint danh sách bài tập

Câu hỏi ① ③

Program.cs · phần thêm mới

```
var assignments = new List<AssignmentDto> {
    new(1, "Bai tap REST API", "Thiet ke endpoint...", true),
    new(2, "Bai tap Client", "Viet console client...", true)
};

app.MapGet("/api/assignments", () => Results.Ok(assignments));

app.Run();
record AssignmentDto(int Id, string Title, string Description, bool IsOpen);
record ApiError(string Code, string Message);
```

Test ngay

```
curl.exe -i localhost:5000/api/assignments
```

```
HTTP/1.1 200 OK

[{"id":1,
  "title":"Bai tap REST API",
  "description":"Thiet ke...",
  "isOpen":true},
 {"id":2, ...}]
```

`record`

Một dòng có đủ thuộc tính, bất biến: hợp với dữ liệu truyền đi

List trong bộ nhớ

Tập trung vào HTTP; dữ liệu mất khi tắt API

`/api/assignments`

URI là **danh từ số nhiều**, không có động từ

Hỏi lớp: C# viết IsOpen, sao JSON lại là isOpen?

Bước 4: chi tiết một bài tập, 200 nếu có, 404 kèm ApiError nếu không

Câu hỏi ② ④

Program.cs · thêm sau endpoint danh sách

```
app.MapGet("/api/assignments/{assignmentId:int}", (int assignmentId) => {
    var assignment = assignments.FirstOrDefault(item => item.Id == assignmentId);
    return assignment is null
        ? Results.NotFound(new ApiError("ASSIGNMENT_NOT_FOUND", "Khong tim thay bai tap."))
        : Results.Ok(assignment);
});
```

{assignmentId:int}

Tham số tuyến + ràng buộc kiểu: giá trị trên URL tự gán vào `int assignmentId`

:int

Gọi `/api/assignments/abc` thì tuyến không khớp, hàm xử lý **không chạy**

NotFound(new ApiError(..))

404 = không tồn tại; body cho **lý do** bằng mã máy đọc được

Tham số kiểu đơn giản (int, Guid) lấy từ URI; kiểu phức tạp lấy từ body.

Test bước 4: cùng là 404 nhưng một cái do handler trả, một cái do framework

Câu hỏi ② ④

4a `curl.exe -i …/api/assignments/1`

```
HTTP/1.1 200 OK

{"id":1,"title":"Bai tap
 REST API",...}
```

Tìm thấy

4b `curl.exe -i …/api/assignments/999`

```
HTTP/1.1 404 Not Found

{"code":
 "ASSIGNMENT_NOT_FOUND",
 "message":"Khong tim
 thay bai tap."}
```

Hàm xử lý chạy, trả 404 có body

4c `curl.exe -i …/api/assignments/abc`

```
HTTP/1.1 404 Not Found
Content-Length: 0

(body rỗng)
```

Framework trả: không tuyến nào khớp

Hỏi lớp: 4b và 4c đều 404. Khác nhau ở đâu? Máy khách xử lý được cái nào bằng code?

Bước 5: hai DTO cho cùng một lượt nộp vì máy khách không được tự điền Id, trạng thái

Câu hỏi ③

ĐẦU VÀO · máy khách gửi

```
record CreateSubmissionRequest(
    string StudentCode,
    string RepositoryUrl);
```

ĐẦU RA · máy chủ trả về

```
record SubmissionDto(
    Guid Id,
    int AssignmentId,
    string StudentCode,
    string RepositoryUrl,
    string Status,
    DateTimeOffset ReceivedAt);
```

Màu cam = trường **máy chủ tự sinh**: Id, AssignmentId (lấy từ URI), Status, ReceivedAt

Không tin máy khách

Nếu dùng chung một lớp, máy khách có thể gửi `"status": "graded"`

Id do máy chủ sinh

`Guid.NewGuid()`: khó đoán, không trùng giữa các máy

assignmentId trên URI

Lượt nộp là tài nguyên con; body chỉ chứa dữ liệu của lượt nộp

DTO đầu vào chỉ chứa những gì máy khách được phép quyết định.

Bước 5: endpoint POST kiểm tra theo thứ tự tồn tại → còn mở → hợp lệ, rồi trả 201

Câu hỏi ② ③ ④

```
app.MapPost("/api/assignments/{assignmentId:int}/submissions",
  (int assignmentId, CreateSubmissionRequest request) => {
    var assignment = assignments.FirstOrDefault(i => i.Id == assignmentId);
    if (assignment is null)
        return Results.NotFound(new ApiError("ASSIGNMENT_NOT_FOUND", ...));
    if (!assignment.IsOpen)
        return Results.BadRequest(new ApiError("ASSIGNMENT_CLOSED", ...));
    if (string.IsNullOrWhiteSpace(request.StudentCode) ||
        string.IsNullOrWhiteSpace(request.RepositoryUrl))
        return Results.BadRequest(new ApiError("INVALID_SUBMISSION", ...));

    var submission = new SubmissionDto(Guid.NewGuid(), assignmentId,
        request.StudentCode.Trim(), request.RepositoryUrl.Trim(),
        "received", DateTimeOffset.UtcNow);
    submissions.Add(submission);
    return Results.Created($"/api/submissions/{submission.Id}", submission);
  });
```

1 **Bài tập có tồn tại?**

`404 ASSIGNMENT_NOT_FOUND`

2 **Bài tập còn mở?**

`400 ASSIGNMENT_CLOSED`

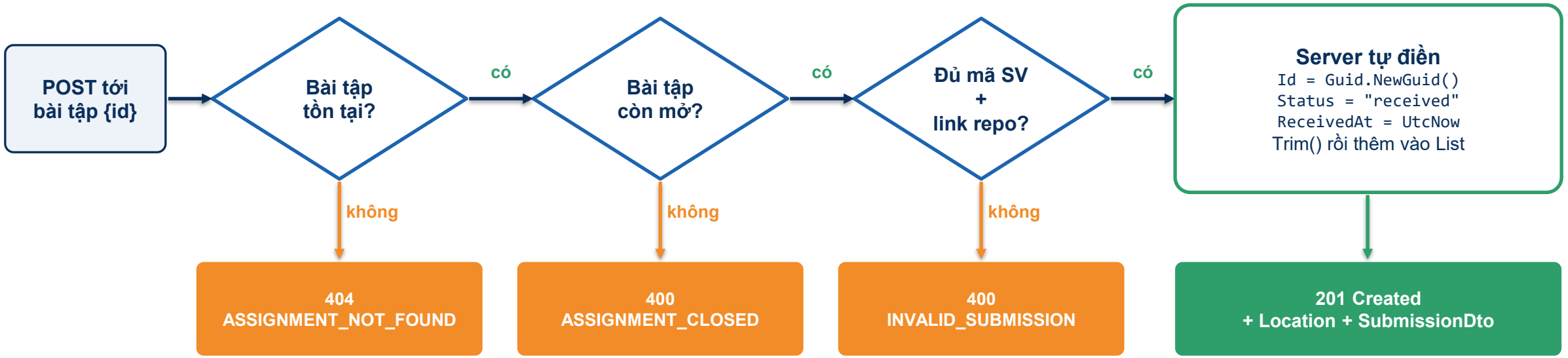
3 **Body hợp lệ?**

`400 INVALID_SUBMISSION`

4 **Tạo, lưu, trả về**

`201 Created + Location`

POST nộp bài: 3 cổng kiểm tra



Trong code là 3 khối if … return:

```
if (assignment is null)            return Results.NotFound(new ApiError("ASSIGNMENT_NOT_FOUND", …));
if (!assignment.IsOpen)             return Results.BadRequest(new ApiError("ASSIGNMENT_CLOSED", …));
if (string.IsNullOrWhiteSpace(…))   return Results.BadRequest(new ApiError("INVALID_SUBMISSION", …));
```

Mỗi "if … return" trong code = một dòng lỗi trong bảng hợp đồng.

Test bước 5: đặt body JSON vào tệp để curl chạy giống nhau trên mọi shell

Câu hỏi ④

sv001.json

```
{ "studentCode": "SV001",
  "repositoryUrl": "https://example.com/sv001/rest-api" }
```

rong.json

```
{ "studentCode": "",
  "repositoryUrl": "" }
```

Mẫu lệnh POST (gõ trên một dòng)

```
curl.exe -i -X POST http://localhost:5000/api/assignments/1/submissions
  -H "Content-Type: application/json" -d "@sv001.json"
```

`-X POST`

Chọn phương thức

`-H "Content-Type: ..."`

Báo body là JSON; thiếu → 415

`-d "@sv001.json"`

Đọc body từ tệp, khỏi lo thoát dấu nháy

`-i`

Hiện mã trạng thái + header

Cách khác: mở `requests.http` trong VS Code, bấm **Send Request** (bước 8).

Thử thách: đoán mã trạng thái của 8 yêu cầu trước khi bấm gửi

Câu hỏi ④

5a POST bài tập 1, body `sv001.json` ?

5b POST bài tập 1, body `rong.json` ?

5c POST bài tập **999**, body `sv001.json` ?

5d Như 5a nhưng **bỏ** `-H "Content-Type..."` ?

5e Body JSON hỏng: `{"studentCode":"SV001",` ?

5f Body thiếu trường: `{"studentCode":"SV001"}` ?

5g `PUT /api/assignments/1` ?

5h Đặt bài tập 2 `IsOpen = false`, POST vào bài 2 ?

NHIỆM VỤ ***Ghi đáp án ra giấy trong 2 phút. Đúng từ 6/8 là đã hiểu mã trạng thái.***

Đáp án: 5 ca do handler trả có ApiError, 3 ca do framework trả không có

Câu hỏi ④

Ca	Yêu cầu	Kết quả thật	Ai trả	Ý nghĩa
5a	POST bài 1, `sv001.json`	**201** + `Location`	Handler	Tạo thành công, có địa chỉ đọc lại
5b	POST bài 1, `rong.json`	**400** `INVALID_SUBMISSION`	Handler	Máy khách gửi sai, sửa body rồi gửi lại
5c	POST bài 999	**404** `ASSIGNMENT_NOT_FOUND`	Handler	Tài nguyên cha không tồn tại
5f	Body thiếu `repositoryUrl`	**400** `INVALID_SUBMISSION`	Handler	Trường thiếu thành null, handler bắt được
5h	POST bài tập đã đóng	**400** `ASSIGNMENT_CLOSED`	Handler	Đúng cú pháp, sai quy tắc nghiệp vụ
5d	Thiếu `Content-Type`	**415**, body rỗng	Framework	Không biết body là JSON, hàm không chạy
5e	JSON hỏng	**400**, body text	Framework	Không đọc được body
5g	`PUT /api/assignments/1`	**405**, `Allow: GET`	Framework	URI có, phương thức không hỗ trợ

Lỗi do handler trả luôn có ApiError; máy khách đọc được bằng một hàm duy nhất.

Bước 6: header Location dẫn tới endpoint đọc lại lượt nộp

Câu hỏi ② ④

Program.cs · thêm trước app.Run()

```
app.MapGet("/api/submissions/{submissionId:guid}", (Guid submissionId) => {
    var submission = submissions.FirstOrDefault(item => item.Id == submissionId);
    return submission is null
        ? Results.NotFound(new ApiError("SUBMISSION_NOT_FOUND", "Khong tim thay luot nop."))
        : Results.Ok(submission);
});
```

6a Chạy 5a, chép `Location`, rồi `GET` địa chỉ đó

`200 OK + đúng lượt nộp`

6b `GET /api/submissions/00000000-0000-...`

`404 SUBMISSION_NOT_FOUND`

6c Tắt API, chạy lại, gọi lại 6a với id cũ

`404 SUBMISSION_NOT_FOUND`

Hỏi lớp (6c): vì sao lượt nộp "biến mất"? Chạy 2 bản API cùng lúc thì sao?

Bước 7, nấc 1: máy khách lấy danh sách bài tập bằng HttpClient

Câu hỏi ⑤

src/Submission.Client/Program.cs

```
using System.Net.Http.Json;

var apiBaseUrl = args.Length > 0 ? args[0]
                                 : "http://localhost:5000";
using var http = new HttpClient {
    BaseAddress = new Uri(apiBaseUrl) };

var assignments = await http
    .GetFromJsonAsync<List<AssignmentDto>>("/api/assignments");
foreach (var a in assignments ?? [])
    Console.WriteLine($"- {a.Id}: {a.Title}");

record AssignmentDto(int Id, string Title, string Description, bool IsOpen);
```

`args[0]`

Địa chỉ API từ dòng lệnh, không gán cứng

`BaseAddress`

Đặt địa chỉ gốc một lần, sau chỉ ghi đường dẫn

`GetFromJsonAsync`

Gửi GET, đọc JSON thành record trong một dòng

`record` ở Client

Khai báo lại: chỉ chia sẻ **hợp đồng JSON**

Terminal 2 · dấu -- chuyển tham số cho chương trình máy khách

```
dotnet run --project src/Submission.Client -- http://localhost:5000
```

Cái bẫy: GetFromJsonAsync ném ngoại lệ khi gặp 404 và vứt bỏ ApiError

Câu hỏi ⑤

Thử tạm đổi đường dẫn

```
var a = await http.GetFromJsonAsync<AssignmentDto>("/api/assignments/999");
```

Kết quả thật: chương trình sập

```
Unhandled exception. System.Net.Http.HttpRequestException:
  Response status code does not indicate success: 404 (Not Found).
```

GetFromJsonAsync

- Ném ngoại lệ khi mã 4xx/5xx
- Mất body `ApiError`
- Chỉ dùng khi chắc chắn thành công

GetAsync / PostAsJsonAsync

- Trả `HttpResponseMessage`, **không** ném
- Tự xem `StatusCode`
- Lỗi thì đọc thành `ApiError`

Quy tắc vàng: xem mã trạng thái trước, đọc body sau.

Bước 7, nấc 2: nộp bài, kiểm tra 201, lỗi thì đọc ApiError bằng một hàm chung

Câu hỏi ⑤

```
// tiếp ngay sau vòng foreach ở nấc 1
var selectedAssignmentId = assignments?.FirstOrDefault()?.Id ?? 1;

var createResponse = await http.PostAsJsonAsync(
    $"/api/assignments/{selectedAssignmentId}/submissions",
    new CreateSubmissionRequest("SV001", "https://example.com/..."));

if (createResponse.StatusCode != HttpStatusCode.Created) {
    await PrintApiErrorAsync(createResponse);
    return;
}
var created = await createResponse.Content
    .ReadFromJsonAsync<SubmissionDto>();

static async Task PrintApiErrorAsync(HttpResponseMessage r) {
    if (r.IsSuccessStatusCode) return;
    var error = await r.Content.ReadFromJsonAsync<ApiError>();
    Console.WriteLine($"{error?.Code}: {error?.Message}");
}
```

`PostAsJsonAsync`

Tự đặt `Content-Type: application/json`, **không** ném khi 4xx

`!= Created`

Xem mã trước: thành công đọc `SubmissionDto`, lỗi đọc `ApiError`

`PrintApiErrorAsync`

Một hàm cho mọi lỗi, nhờ API trả **cùng một dạng** lỗi

Code đầy đủ: `src/Submission.Client/Program.cs`

Kết quả máy khách: không sập khi gặp 400, và phân biệt lỗi mạng với lỗi HTTP

Câu hỏi ⑤

Kết quả thật

```
Danh sach bai tap dang mo:
- 1: Bai tap REST API
- 2: Bai tap Client

Da tao luot nop: d8e5f642-000b-...
Trang thai ban dau: received
Doc lai luot nop: SV001 - https://...

Thu gui yeu cau loi: 400 BadRequest
INVALID_SUBMISSION: Thieu ma sinh vien
  hoac kho ma nguon.
```

7a — Tắt API rồi chạy Client

`Connection refused`: lỗi **mạng**, không có phản hồi

7b — Chạy Client với cổng sai 5999

Như 7a: địa chỉ dịch vụ là cấu hình

7c — Thêm bài tập 3 vào API

Client in thêm dòng mà **không sửa code**

Lỗi mạng (không có phản hồi) khác lỗi HTTP (có phản hồi 4xx).

Bước 8: requests.http lưu mọi ca test, bấm Send Request là thấy phản hồi

Câu hỏi ④

requests.http (trích)

```
@api = http://localhost:5000

### Bài tập không tồn tại → 404
GET {{api}}/api/assignments/999

### Nộp bài cho bài tập 1 → 201
# @name createSubmission
POST {{api}}/api/assignments/1/submissions
Content-Type: application/json

{ "studentCode": "SV001", "repositoryUrl": "https://..." }

### Đọc lại lượt nộp vừa tạo → 200
GET {{api}}/api/submissions/
    {{createSubmission.response.body.$.id}}
```

`@api = ...`

Biến: đổi cổng một chỗ là đổi cả tệp

`###`

Ngăn cách request; tiêu đề ghi luôn **mã mong đợi**

`Dòng trống`

Bắt buộc giữa header và body

`# @name + {{...id}}`

Lấy `id` từ phản hồi trước; gửi POST trước

Tổng hợp: 8 mã trạng thái gặp hôm nay, ai trả và máy khách đọc được gì

Mã	Tên	Ca	Ai trả	ApiError?	Thông điệp cho SV
200	OK	3, 4a, 6a	Handler	– (dữ liệu)	Đọc thành công
201	Created	5a, 9b	Handler	– + `Location`	Tạo thành công, có địa chỉ đọc lại
204	No Content	9d	Handler	Không	Thành công, cố ý không trả gì
400	Bad Request	5b, 5f, 5h, 9a	Handler	Có	Máy khách gửi sai, sửa rồi gửi lại
404	Not Found	4b, 5c, 6b, 9e	Handler	Có	Tài nguyên không tồn tại
409	Conflict	9c	Handler	Có	Hợp lệ nhưng xung đột trạng thái
404	Not Found	4c (`/abc`)	Framework	Không	Đường dẫn không khớp tuyến nào
405	Method Not Allowed	5g	Framework	Không	URI đúng, phương thức sai
415	Unsupported Media Type	5d	Framework	Không	Thiếu `Content-Type`
–	Không có phản hồi	7a	Mạng	–	Lỗi mạng khác lỗi HTTP

2xx thành công · 4xx lỗi phía máy khách · 5xx lỗi phía máy chủ

Nộp bài Bài 2: làm trong fork repo lớp, mở một Pull Request

Làm trong <Họ tên - MSSV>/Bai2 → Commit mỗi bước → Push ở checkpoint → Mở PR vào repo lớp

Mã nguồn

API 4 endpoint + client chuỗi 4 bước, build không lỗi. Tiêu đề PR: Bai2 - MSSV - Họ tên.

requests.http

Đủ 7 request, bao phủ 200, 201, 400, 404.

README.md (nhật ký)

Lệnh đã chạy, ảnh kết quả, bảng phá thử, trả lời 3 câu hỏi.

Ba câu hỏi giải thích (ghi vào README.md)

1. Vì sao URI dùng danh từ, còn hành động do phương thức HTTP đảm nhận?
2. “Stateless” thể hiện ở đâu trong code đã viết?
3. Vì sao client và server mỗi bên khai báo DTO riêng mà vẫn khớp nhau?

TỔNG KẾT

Chốt bài và chuẩn bị Bài 3

Thông điệp chính: hợp đồng rõ ràng (URI, phương thức, mã trạng thái, DTO) giúp hai tiến trình hiểu nhau.

chốt buổi | tóm tắt | giới hạn | ôn tập | Bài 3

04

Chốt buổi: năm lỗi thiết kế cần tránh và câu trả lời cho 3 câu hỏi đầu giờ

NĂM LỖI HAY GẶP KHI TỰ THIẾT KẾ

1. URI có động từ: `POST /api/borrowBook` → `POST /api/books/{id}/loans`
2. Đưa `bookId` vào body thay vì lên URI
3. DTO đầu vào có `Id` hoặc `Status` cho máy khách tự điền
4. Dùng 400 cho mọi lỗi, không tách 404 với 409
5. Mỗi lỗi trả một kiểu chuỗi, máy khách không đọc được bằng code

BA CÂU HỎI ĐẦU GIỜ

1. **Nói chuyện bằng gì?**

   Yêu cầu và phản hồi HTTP theo một **hợp đồng API**.

2. **Thiết kế bắt đầu từ đâu?**

   Từ **danh từ**, qua 5 câu hỏi, rồi mới code.

3. **Biết thành công hay thất bại?**

   Mã trạng thái + một dạng **ApiError** thống nhất.

1

Hai chương trình chạy ở hai nơi nói chuyện với nhau bằng gì?

Đáp án (mở cuối buổi)

Bằng yêu cầu và phản hồi HTTP, theo một **hợp đồng** đã thống nhất.

2

Thiết kế một API bắt đầu từ đâu?

Đáp án (mở cuối buổi)

Từ các **danh từ** của bài toán, không phải từ code.

3

Làm sao máy khách biết yêu cầu thành công hay thất bại?

Đáp án (mở cuối buổi)

Nhờ **mã trạng thái** và một dạng lỗi **thống nhất**.

Hiểu = giải thích được · Biết = chạy lại được · Vận dụng = thiết kế được cho ứng dụng khác

Tóm tắt: sáu ý mang theo sau Bài 2

1 REST là request–response đồng bộ

Trên HTTP, cho thao tác có ranh giới rõ.

2 Thông điệp HTTP là hợp đồng

Phương thức, URI, header, body, mã trạng thái.

3 URI là tài nguyên, method là hành động

Đường dẫn định danh; query để lọc, phân trang.

4 Stateless

Mỗi yêu cầu tự đủ thông tin; không phải "không lưu dữ liệu".

5 DTO ở biên, ApiError cho mọi lỗi

Không lộ nội bộ; client chỉ cần một hàm đọc lỗi.

6 Client kiểm tra mã trước khi đọc

Lỗi là tình huống bình thường của hệ phân tán.

Mỗi giới hạn hôm nay là lý do học một bài sau

Code hôm nay	Giới hạn	Phát triển ở
`new List<SubmissionDto>()`	Tắt API là mất; List không an toàn khi nhiều yêu cầu cùng lúc	Lưu CSDL (mở rộng ngoài đề cương, gợi ý EF Core + SQLite)
`Status: "received"`	Mãi không đổi: chưa ai chấm, client phải tự hỏi lại	B3 SignalR đẩy processing → graded; B6 worker chấm thật
`return Results.Created(…)`	Nếu chấm ngay trong POST mất 5 phút, client phải chờ	B5 gọi gRPC chấm; B6 trả 202 Accepted + hàng đợi
`GET từng tài nguyên`	Màn hình GV cần nhiều yêu cầu, thừa/thiếu trường	B4 GraphQL: hỏi đúng trường cần
`StudentCode client tự khai`	Ai cũng nộp hộ được SV001	Xác thực (mở rộng ngoài đề cương, gợi ý JWT)
`http://localhost:5000`	Chỉ chạy trên một máy, một tiến trình	B8 cấu hình môi trường, Docker Compose, health check

Giới hạn hôm nay là CỐ Ý: lát cắt nhỏ chạy được trước, rồi mới lớn dần.

Câu hỏi ôn tập và thảo luận

ÔN TẬP (chọn từ câu hỏi cuối chương)

1. Một yêu cầu HTTP gồm những phần nào? Vai trò từng phần?
2. Vì sao không chỉ đọc body JSON mà bỏ qua mã trạng thái?
3. Vì sao POST /api/assignments/{id}/submissions rõ nghĩa hơn POST /api/submissions?
4. Khi nào đặt dữ liệu vào đường dẫn, khi nào vào chuỗi truy vấn?
5. Stateless khác gì “server không được lưu dữ liệu”?
6. Vì sao tạo thành công nên trả 201 kèm đường dẫn đọc lại?

THẢO LUẬN

- Lượt nộp là tài nguyên con của bài tập: ưu điểm, hạn chế so với đặt hết dưới /api/submissions?
- Nếu DTO phản hồi trả toàn bộ mô hình nội bộ, rủi ro nào xuất hiện khi hệ thống lớn lên?
- Khai thác API của nhóm khác: cần những thông tin gì?

Tài liệu tham khảo

[1] Giáo trình Phát triển ứng dụng phân tán, Chương 2 “Xây dựng RESTful API cho tương tác máy khách–máy chủ”, mục 2.1–2.3, Mã nguồn 2.1–2.4 (nguồn chính).

[2] R. T. Fielding (2000). Architectural Styles and the Design of Network-based Software Architectures. Luận án TS, University of California, Irvine, chương 5.

[3] IETF RFC 9110 – HTTP Semantics (2022): phương thức an toàn và lũy đẳng (mục 9.2), mã trạng thái (mục 15).

[4] IETF RFC 9457 – Problem Details for HTTP APIs (2023).

[5] Microsoft Learn. Parameter binding in Minimal API apps; Routing in ASP.NET Core (route constraints). ASP.NET Core 10.0.

[6] Microsoft Learn. What’s new in ASP.NET Core in .NET 10: Validation support in Minimal APIs.

[7] Microsoft Learn. HttpClientJsonExtensions.GetFromJsonAsync; Invoke-WebRequest. H. Mao, REST Client for Visual Studio Code.

Bài 3: làm sao client biết ngay khi lượt nộp đổi trạng thái?

Lượt nộp đang "received". Làm sao client biết ngay khi nó chuyển sang "processing", rồi "graded"?

Cách đã biết · polling bằng REST

```
// Hỏi mãi, kể cả khi không có gì mới
while (true) {
    var s = await http.GetFromJsonAsync
        <SubmissionDto>(url);
    await Task.Delay(2000);
}
```

Bài 3 · server đẩy cập nhật



Hub `/hubs/submissions` thêm vào **chính Submission.Api** của Bài 2: server chủ động báo cho client khi trạng thái đổi.

Chuẩn bị trước buổi học

- Hoàn thành bài tập 1–4 và mở PR Bài 2
- Đọc trước mục 3.1 giáo trình: truyền thông thời gian thực
- Giữ nguyên Submission.Api để Bài 3 sửa tiếp

PHỤ LỤC · TRA CỨU VÀ MỞ RỘNG

Tra cứu lỗi, mở rộng và bài về nhà

Dùng khi thực hành gặp lỗi, hoặc giao làm thêm sau buổi học.

lỗi Catalog | lỗi EduSubmit | 409 / 204 | vận dụng | đổi miền

Gặp lỗi? Tra bảng này trước khi hỏi

tra cứu

Triệu chứng	Nguyên nhân	Cách sửa
`Couldn't find a project to run`	Quên --project hoặc đứng sai thư mục	cd về CatalogApi, thêm --project src/Catalog.Api
`Failed to bind ... 5100: address already in use`	Còn máy chủ cũ đang chạy	Ctrl+C ở terminal cũ hoặc đóng hẳn terminal đó
`CS8803: Top-level statements must precede...`	record viết giữa tệp	Chuyển mọi record xuống cuối, sau app.Run();
`CS0246: 'ProductDto' could not be found`	Quên khai báo record (thường ở máy khách)	Thêm record ProductDto(...) cuối Program.cs
`Sửa code mà kết quả không đổi`	Chưa khởi động lại máy chủ	Ctrl+C rồi dotnet run lại
`Máy khách in tên trống`	Tên thuộc tính DTO không khớp JSON	So từng tên với DTO máy chủ
`400 khi gửi JSON bằng -d '{...}'`	PowerShell làm hỏng dấu nháy	Dùng -d "@tep.json" hoặc requests.http
`REST Client gửi body rỗng`	Thiếu dòng trống giữa header và body	Thêm một dòng trống

Đọc dòng lỗi đầu tiên, tìm mã lỗi (CSxxxx, NETSDKxxxx) hoặc mã trạng thái.

Sáu lỗi hay gặp khi chạy ví dụ và cách xử lý trong 1 phút

Hiện tượng	Nguyên nhân	Cách xử lý
`address already in use` (cổng 5000)	Còn một bản API đang chạy	Ctrl+C terminal cũ, hoặc đổi `--urls ...:5001`
Client báo `Connection refused`	Chưa chạy API hoặc sai cổng	Chạy API trước, xem dòng `Now listening on`
`curl` báo lỗi lạ với `-i`, `-X`	Gọi nhầm `Invoke-WebRequest`	Gõ `curl.exe`
POST luôn nhận 415	Thiếu `Content-Type: application/json`	Thêm `-H`, hoặc dùng `PostAsJsonAsync`
Sửa code mà kết quả không đổi	Chưa khởi động lại API	Ctrl+C rồi chạy lại; `dotnet watch` thì Ctrl+R
Lượt nộp cũ trả 404	API khởi động lại, dữ liệu bộ nhớ mất	Đúng thiết kế (ca 6c)
`NETSDK1045`	Máy cài SDK cũ	Cài .NET 10 SDK

Bước 9 (mở rộng): 409 khi yêu cầu hợp lệ nhưng xung đột với dữ liệu đã có

Câu hỏi ② ④

Program.cs · bản MoRong

```
// chèn SAU khối kiểm tra INVALID_SUBMISSION trong handler POST
var duplicated = submissions.Any(item =>
    item.AssignmentId == assignmentId &&
    item.StudentCode.Equals(request.StudentCode.Trim(), StringComparison.OrdinalIgnoreCase));
if (duplicated)
    return Results.Conflict(new ApiError("DUPLICATE_SUBMISSION", "Sinh vien da nop bai tap nay."));
```

400 Bad Request

Bản thân yêu cầu **sai**: thiếu trường, bài đã đóng. Gửi lại y hệt vẫn sai.

404 Not Found

Tài nguyên **không tồn tại**: bài tập 999.

409 Conflict

Yêu cầu **đúng**, nhưng **xung đột** với dữ liệu đã có: SV đã nộp bài này.

Hỏi lớp: gửi đúng yêu cầu đó vào bài tập 2 thì được mã gì? (201)

Bước 9 (mở rộng): DELETE trả 204 không body; chuỗi 7 ca cho thấy đủ mã

Câu hỏi ② ④

Thêm trước app.Run()

```
app.MapDelete("/api/submissions/{submissionId:guid}",
  (Guid submissionId) => {
    var s = submissions.FirstOrDefault(
        i => i.Id == submissionId);
    if (s is null)
        return Results.NotFound(new ApiError(
            "SUBMISSION_NOT_FOUND", "..."));
    submissions.Remove(s);
    return Results.NoContent();
  });
```

Ca	Yêu cầu	Kết quả
9a	POST bài tập 3 (đã đóng)	**400** CLOSED
9b	POST bài 1, SV002	**201** Created
9c	Gửi lại 9b	**409** DUPLICATE
9d	DELETE lượt nộp 9b	**204** No Content
9e	Gửi lại 9d	**404** NOT_FOUND
9f	Gửi lại 9b sau khi rút	**201** id mới
9g	Chạy Client 2 lần	Lần 2: **409**

Client không sửa dòng nào vẫn in đúng lỗi 409: hợp đồng lỗi thống nhất.

Vận dụng: tự thêm endpoint theo đúng quy tắc REST đã học

vận dụng

1 Lọc theo từ khóa

GET /api/products?keyword=REST. Thêm tham số string? keyword vào handler danh sách (liên kết từ query), lọc bằng Where + Contains.

2 Xóa

DELETE /api/products/{id:int}: 204 No Content khi xóa được, 404 + PRODUCT_NOT_FOUND khi không có. MapDelete, Remove, Results.NoContent().

3 Giới hạn độ dài tên

Tên > 100 ký tự → 400 NAME_TOO_LONG. Kiểm tra này đặt trước hay sau INVALID_PRODUCT? Vì sao?

4 So sánh hai kiểu 404

Gọi /abc và /999, chép lại header và body, giải thích vì sao khác.

5 Đổi miền

Dựng LibraryApi (/api/books) hoặc ClinicApi (/api/appointments) theo đúng 10 bước: 2 DTO + ApiError, 4 endpoint, máy khách 4 nấc.

NHIỆM VỤ **Mỗi bài: viết endpoint → thêm ca test vào requests.http → chạy, ghi mã nhận được**

Nhiệm vụ "Đổi miền" (giao về nhà): thiết kế API cho một ứng dụng khác

1

Trả lời 5 câu hỏi

Bảng ① → ⑤ cho ứng dụng nhóm chọn

2

Viết bảng hợp đồng API

Phương thức, URI, thành công, lỗi

3

Viết requests.http 6 ca

200 · 201 · 400 · 404 · 409 hoặc 204 · 405 hoặc 415

4

Nhóm khá: chạy được

Sao chép Submission.Api, đổi tên, gửi 6 request

Mức	Dấu hiệu quan sát được
Hiểu	Giải thích được 3 câu hỏi đầu giờ; chỉ ra 4 ô HTTP trên một yêu cầu thật ở tab Network
Biết	Chạy lại được EduSubmit; đoán đúng mã trạng thái trước khi bấm Send
Vận dụng	Tự thiết kế đúng: URI không có động từ, con nằm dưới cha, DTO vào không có Id, chọn đúng 400/404/409

NHIỆM VỤ ***Nộp: ảnh bảng hợp đồng + tệp requests.http vào thư mục Bai2 của nhóm.***